

LESSON 19

Bayes Optimal Classification,
Nearest Neighbor Classifiers (kNN)

Clustering (k means).

Main intelligent systems approaches:
Neural Networks, Fuzzy systems, Evolutionary, Hybrid
systems

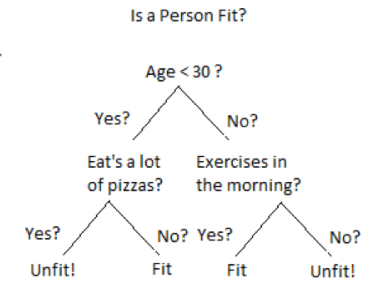
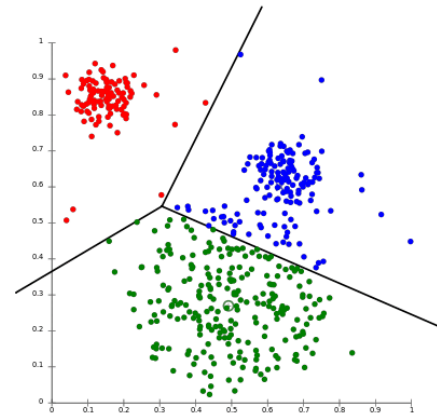


Outline

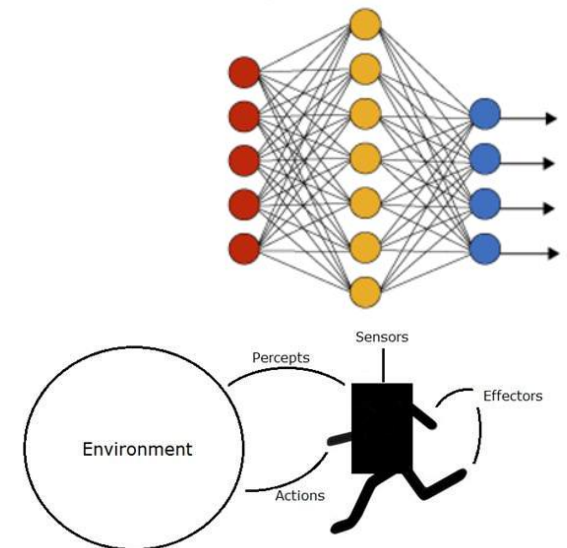
- Bayes Optimal Classification
- Relevance of Classical (non-neural) models
- Eager and Lazy Learning Methods
- Nearest Neighbor Classifiers (kNN)
 - Relevance of the kNN in pattern recognition
 - Definition
 - Problems
 - Speed
 - Curse of dimensionality

Outline (2)

- Clustering (k means)
- Main types of computational intelligence systems
 - Fuzzy systems
 - Evolutionary systems
 - Agents
 - Hybrid systems
- Example of Genetic Parameters Optimization



Simple Neural Network





Bayes Optimal Classification

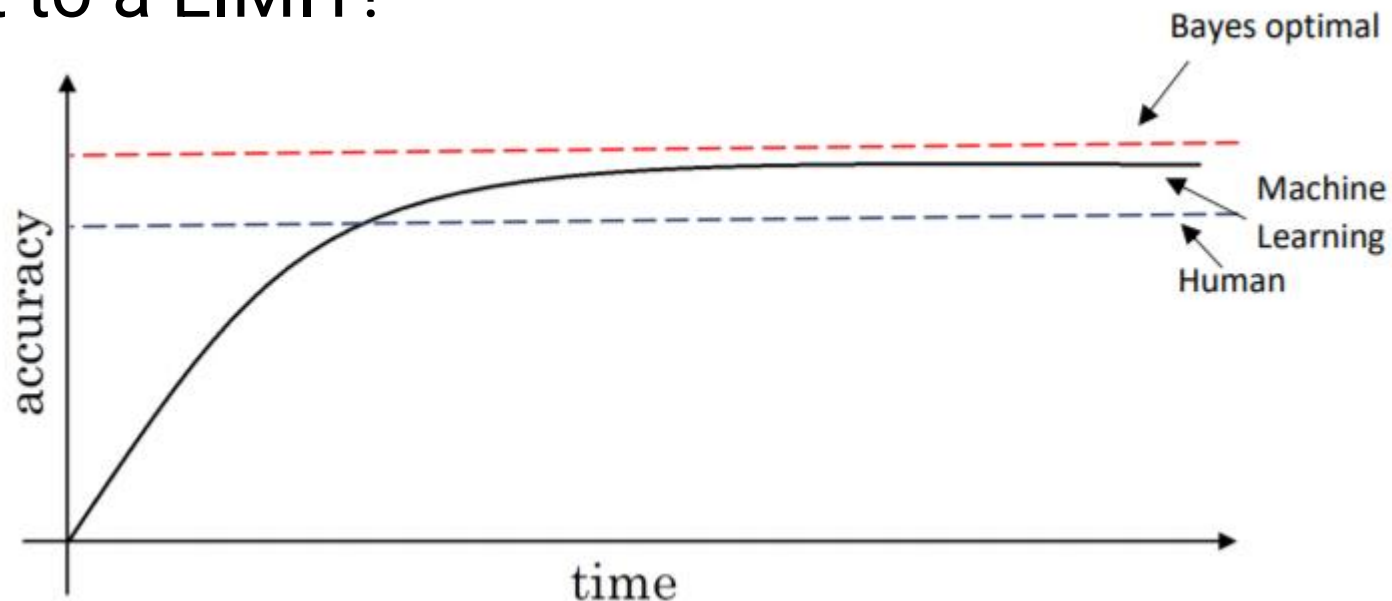
We are chasing the minimal error,
but what is the limit?!


**LETS
ENSURE
ZERO
ERROR
???**

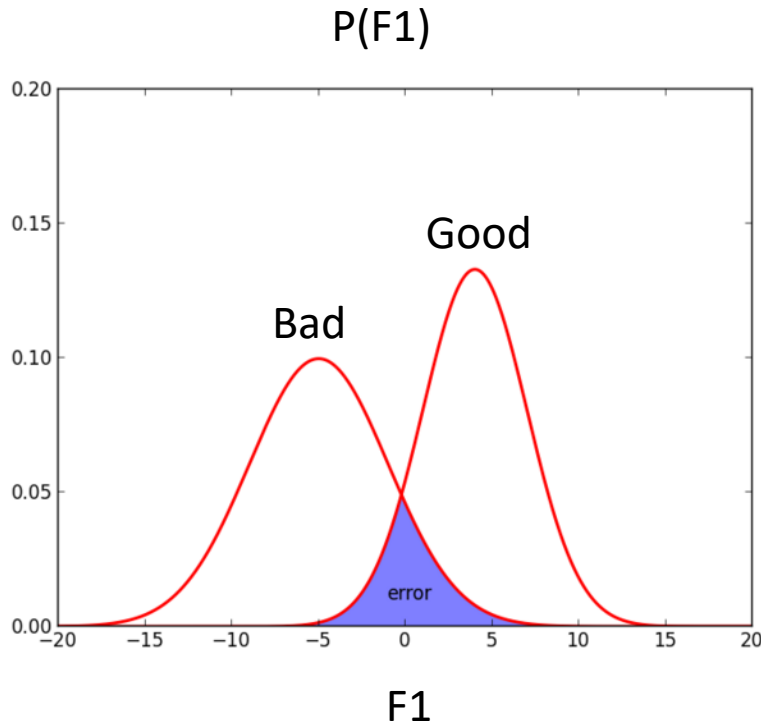
Always chasing the
0% mistakes
can be a mistake!

Bayes optimal limit

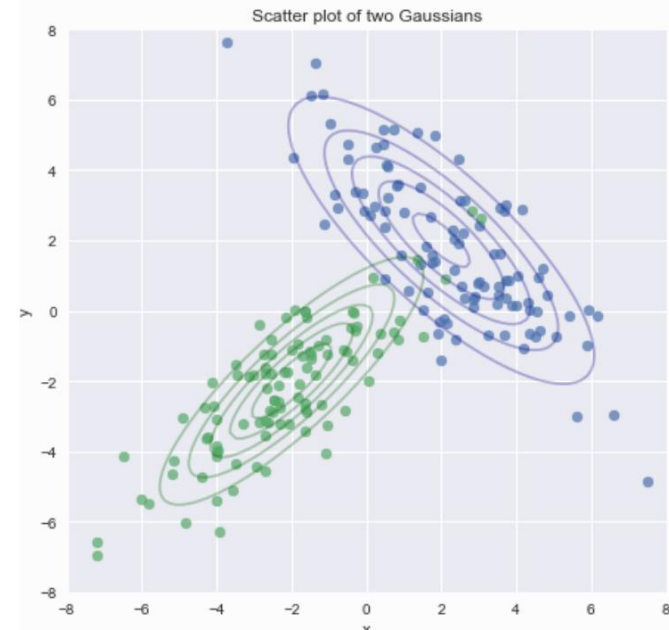
Given a specific problem/dataset, over time, making more attempts and using new techniques (e.g., Deep Learning) the ML algorithms can outperform humans... but to a LIMIT!



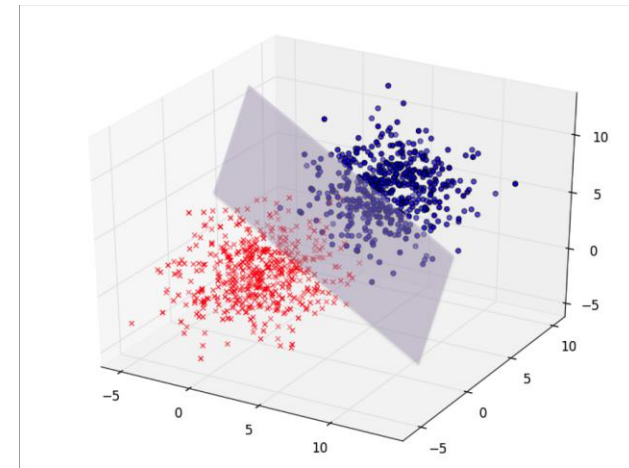
Bayes optimal error



2D

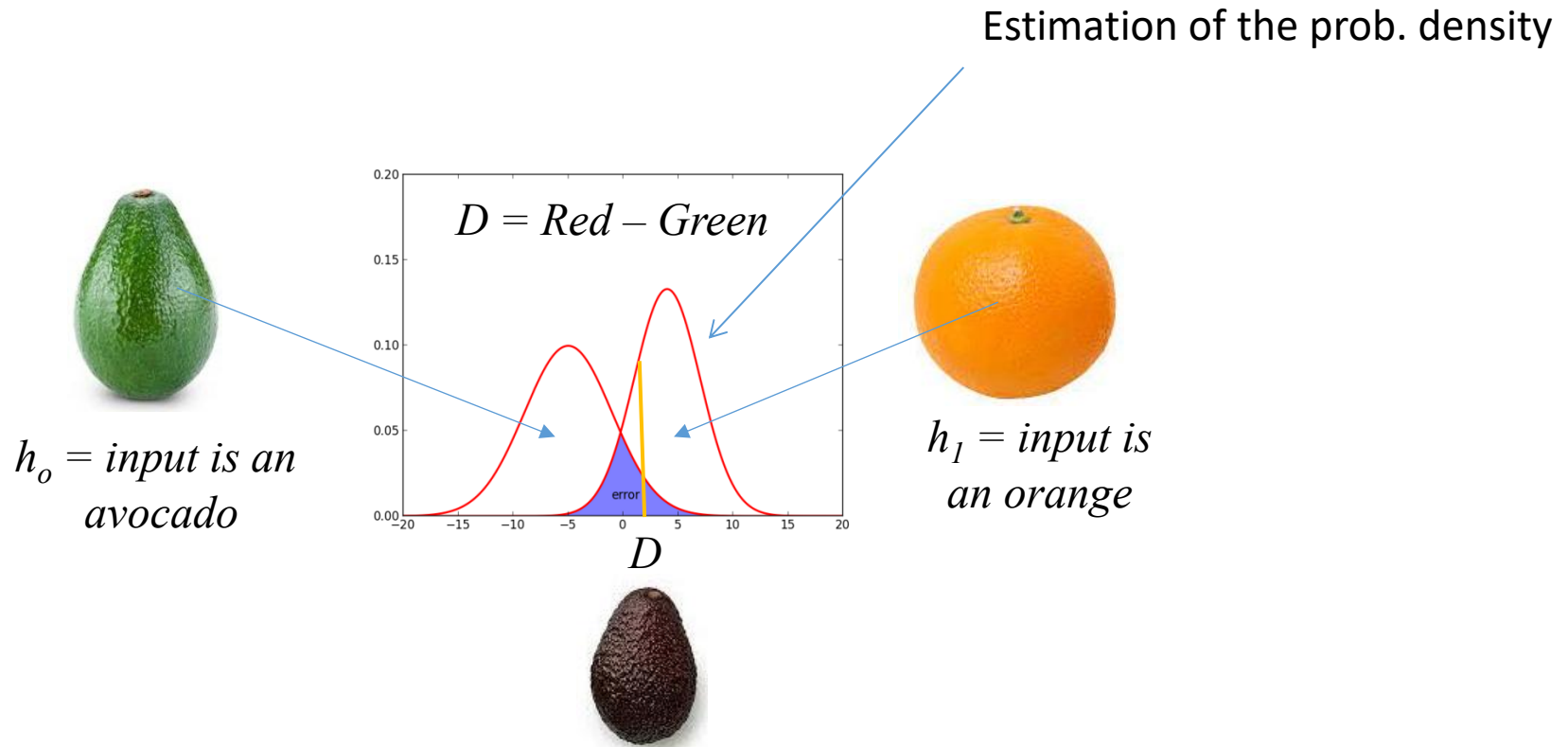


3D



Every time the distributions of the classes are not separated even the best classifier is producing an error related to the overlaps

Bayes Optimal Classifier

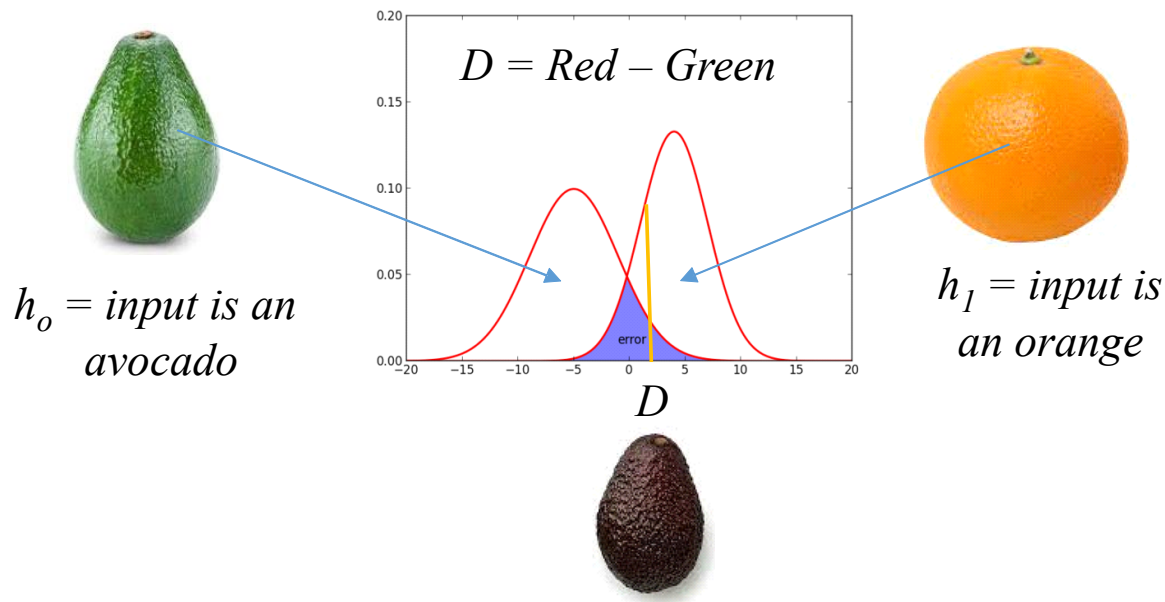


My predictions can be
 $v_o = I \text{ believe it is an avocado}$
 $v_1 = I \text{ believe it is an orange}$

Bayes Optimal Classifier

Intuitively, when observing the features D (e.g., color and weight), we instinctively classify the object as an apple because, in the dataset, most samples with similar values of D are apples.

That is, the probability $P(C = \text{orange} \mid x)$ is higher than $P(C = \text{Avocado} \mid x)$ for the observed data point x .



Bayes Optimal Classifier

Defined as the label produced by the most probable classifier

Output Class

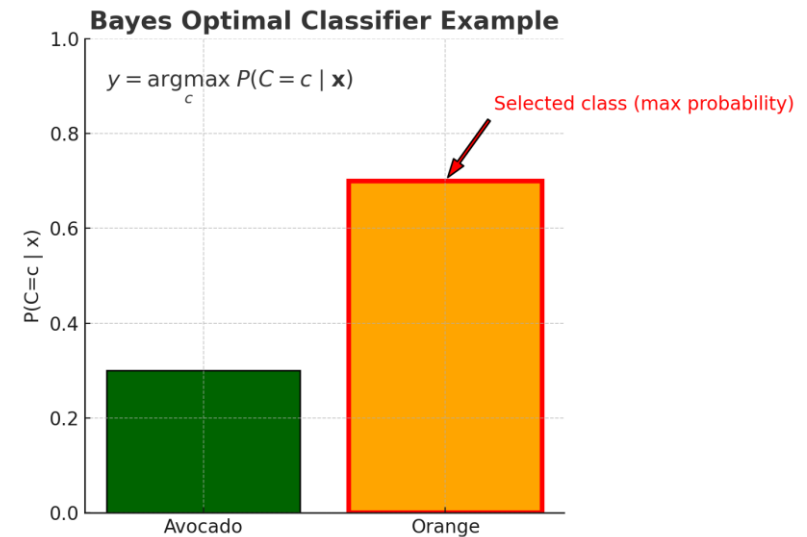
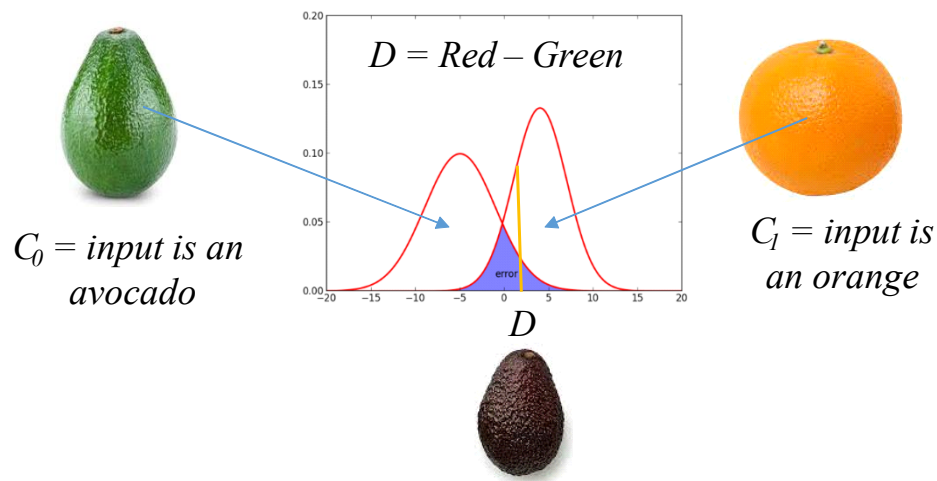
$$\hat{y} = \arg \max_{C_k} P(C_k | X)$$

$$y = \operatorname{argmax}_c P(C=c | D)$$

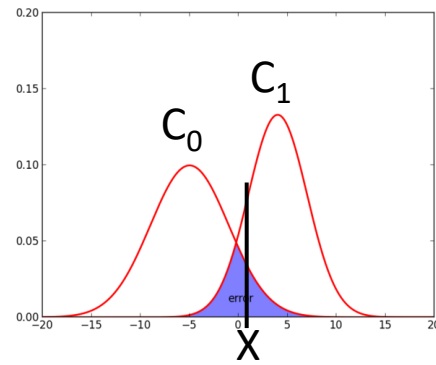
$$= \operatorname{argmax} \{ P(C=\text{Avocado} | D), P(C=\text{Orange} | D) \}$$

$$= \operatorname{argmax} \{ 0.3, 0.7 \}$$

$$= \text{Orange}$$



Bayes Optimal Classification



To classify a data point described by the feature vector

$$X = (x_1, x_2, \dots, x_n)$$

we choose the class with the highest posterior probability:

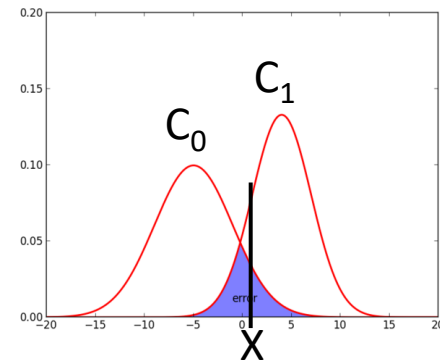
$$\hat{y} = \arg \max_{C_k} P(C_k | X)$$

Using Bayes' theorem:

$$P(C_k | X) = \frac{P(X | C_k)P(C_k)}{P(X)}$$

Since $P(X)$ is the same for all classes, it does not affect the maximization:

$$\hat{y} = \arg \max_{C_k} P(X | C_k)P(C_k)$$



Bayes Optimal Classification

- Defined as the label produced by the most probable classifier

$$\hat{y} = \arg \max_{C_k} P(X | C_k)P(C_k)$$

- Computing real world $P()$ this can be very inefficient or impossible
 - Especially when X is a measure in a high dimensionality space
- You can compute $P()$ but you must know the distributions and have independent features.
- Theoretical concept: No other classification method can outperform this method on average (using the same hypothesis space and prior knowledge).

Minimum possible error can be > 0 !

Naive Bayes Classification

To classify a data point described by the feature vector

$$X = (x_1, x_2, \dots, x_n)$$

we choose the class with the highest posterior probability:

$$\hat{y} = \arg \max_{C_k} P(C_k | X)$$

Using Bayes' theorem:

$$P(C_k | X) = \frac{P(X | C_k)P(C_k)}{P(X)}$$

Since $P(X)$ is the same for all classes, it does not affect the maximization:

$$\hat{y} = \arg \max_{C_k} P(X | C_k)P(C_k)$$

The Naive assumption

$$\hat{y} = \arg \max_{C_k} \underbrace{P(X | C_k) P(C_k)}$$

The difficult term is:

$$P(X | C_k)$$

because it is the joint probability of all features given the class:

$$P(x_1, x_2, \dots, x_n | C_k)$$

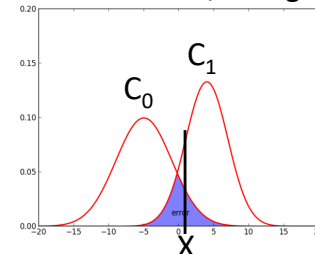
Naive Bayes simplifies this by assuming that the features are conditionally independent given the class:

$$P(X | C_k) = \prod_{i=1}^n P(x_i | C_k)$$

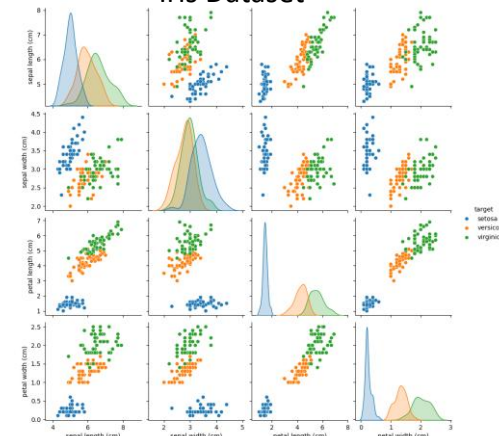
Therefore, the Naive Bayes classifier becomes:

$$\hat{y} = \arg \max_{C_k} P(C_k) \prod_{i=1}^n P(x_i | C_k)$$

Avocado/Orange



Iris Dataset



Naive Bayes Classification

$$\hat{y} = \arg \max_{C_k} P(X | C_k)P(C_k)$$

- From the theory of the Bayes optimal classifier one can build a classifier using the data in the train dataset to build the estimation of the distributions and hence to find a proper separation plane
- Hypothesis: strong independence of the features (rarely satisfied → hence “naïve”)
- Requiring a number of parameters linear in the number of variables in a learning problem.
- Suited with high dimensionality of the inputs.
- In general, the accuracy is good w.r.t. other classical classifier!

Coding the naïve Bayes class.

Step	MATLAB	Python (scikit-learn)
Training	<pre>net = fitcnb(X, Y);</pre>	<pre>from sklearn.naive_bayes import GaussianNB net = GaussianNB().fit(X, Y)</pre>
Prediction	<pre>out = predict(net, X_new);</pre>	<pre>out = net.predict(X_new)</pre>



Classical models

Pro and cons of standard and non-neural models

Classical (non neural) methods are important!

- (Some of them) are simple → Occam's razor
- The learning methods are very well known, and they tend to be present in all ML tools/environments
- They give you a very solid reference about accuracy
- Explainability is higher
 - E.g., in deep learning models explainability is very hard



Examples of classical methods (non neural)

- **Distance-based:**
 - - kNN
- **Tree-based:**
 - - Decision Tree
- **Linear/Parametric:**
 - - Linear, Quadratic, Logistic Classifiers
- **Probabilistic / Distribution-based:**
 - Bayes Optimal Classifier
 - Naïve Bayes Classifier
- **Kernel methods:**
 - - Support Vector Machine, Radial Basis Functions

These models are preferred when interpretability, simplicity, or limited data make deep learning unsuitable.



“Eager” and “Lazy” Learning methods

An important difference to be noted

Eager and Lazy Learning Methods

- **Eager** Learning
 - Explicit description of target function on the whole training set
 - The system tries to construct a general, input-independent target function during training of the system
- **Instance-based** Learning (Lazy)
 - Learning=storing all training instances
 - Classification=assigning target function to a new instance
 - The generalization beyond the training data is delayed until a query is made to the system.
 - Referred to as “**Lazy**” learning

Eager Learning

A general, input-independent target function is created during the training



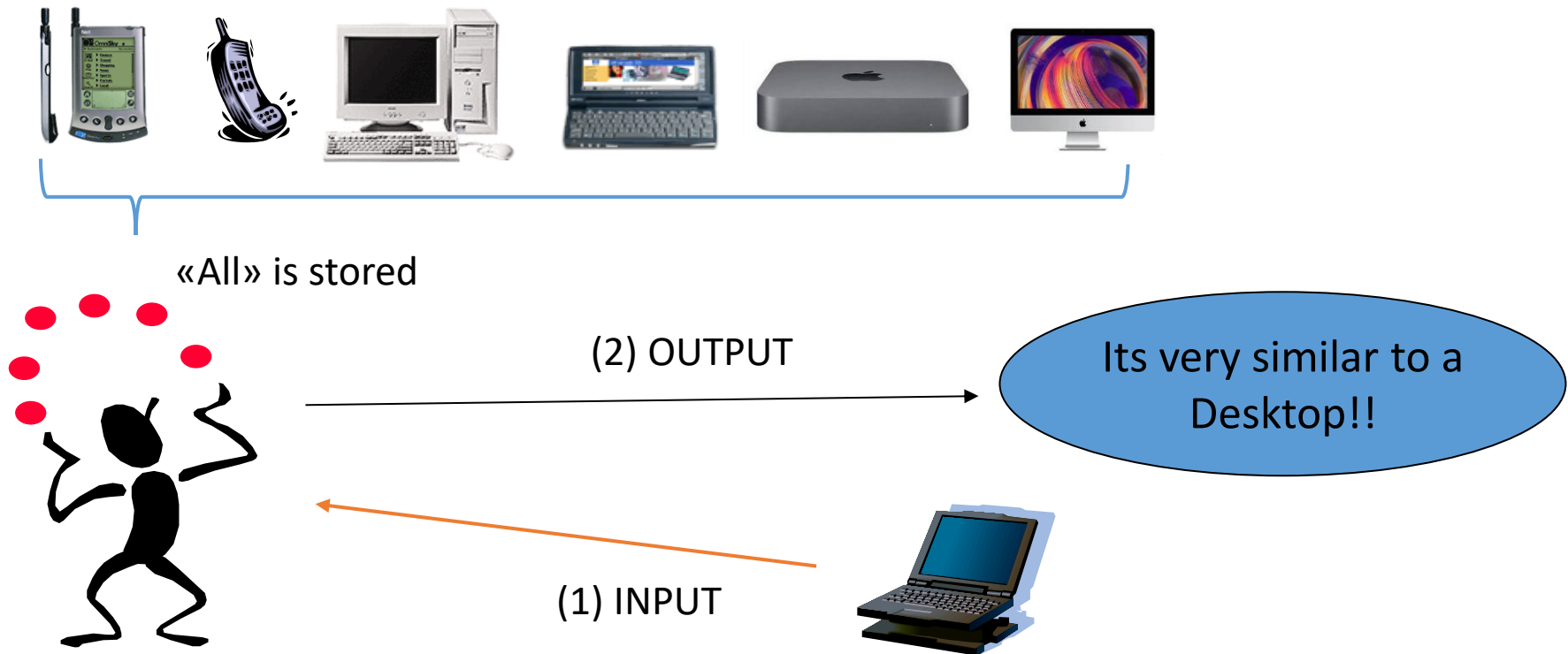
Any random movement
=>It's a mouse

It's a mouse!

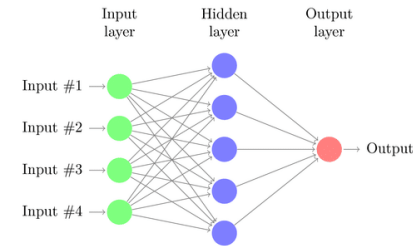
Generalizations and
concepts are produced
during learning

Instance-based Learning (Lazy)

The generalization beyond the training data is delayed until a query is made to the system.



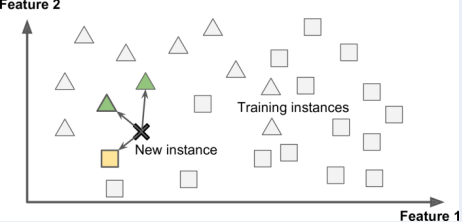
Tradeoff/features



	Training	Recall (deployment)
Eager learners	• Long • #DoF limited (not for deep nets)	• Fast • Low Memory
Instance-based	• Fast • #DoF $\leftrightarrow$ data	• Long • Large Memory req.



Examples

	Non-neural (classic) models	Neural methods
Eager learners	• Decision Trees • Induction and rule-based systems • Support Vector Machines (SVM)	• Feed-forward • Convolutional Neural Networks (CNN)
Instance-based 	• k-Nearest Neighbors algorithm • Weighted Regression • Case-based reasoning	• Kernel Machines • Radial Basis Function Neural Networks (RBFNN)



Classical models: kNN

Nearest Neighbor Classifiers

Nearest Neighbor Classifier: A “must-have”. Why?

- It's a classical classifier not based on neural techniques
- It's deterministic
 - No random initialization (like NN., EVO. algo, ...)
 - Perfect repeatability
- A minimum number of parameters is needed

Nearest Neighbor Classifier: A “must-have”. Why? (2)

- Learning is very simple
- Perfect explainability
 - In term of how is really functioning
 - In term of «why this sample has been classified as bad»
- Present in all libraries
- Starting from kNN you will understand important key point valid also large models such as deep neural networks

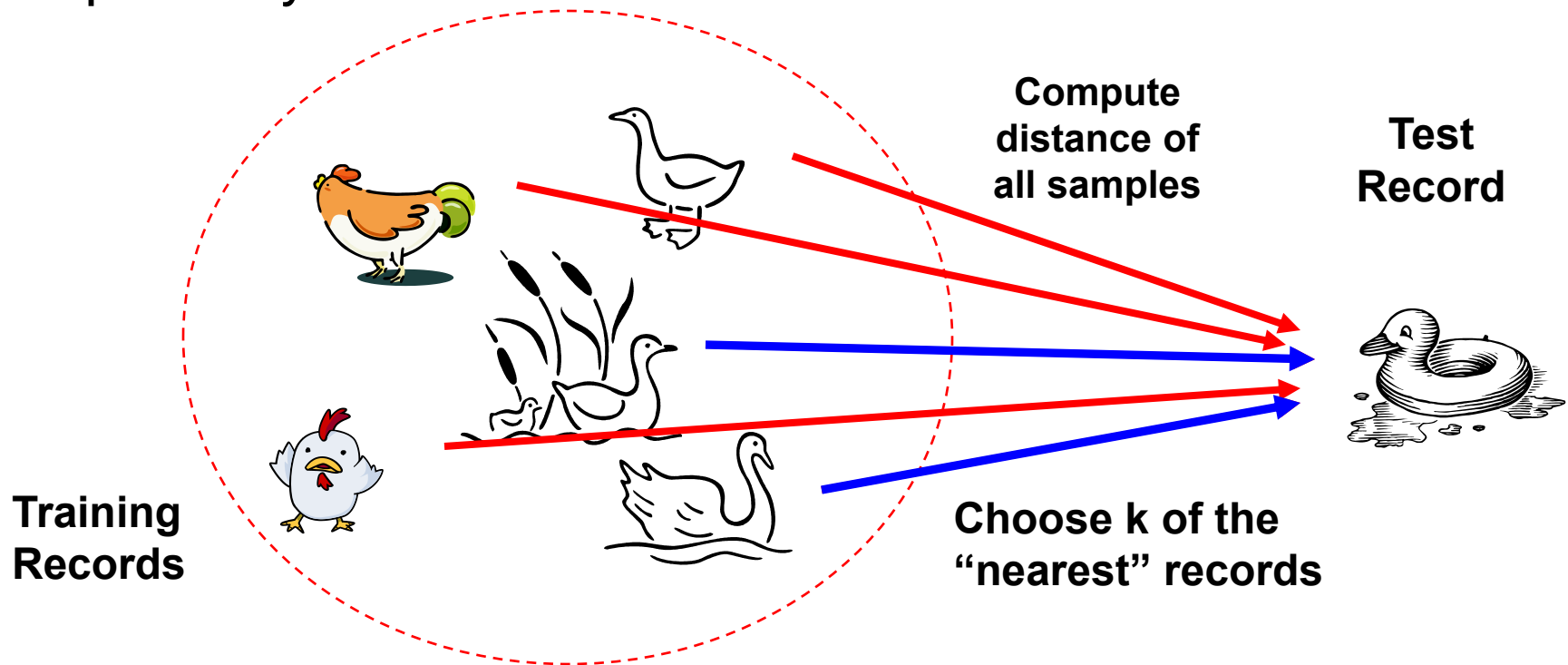
Why study the kNN?

- Because it is very powerful classifier to understand your data
 - if the dataset is not too big... (out-of-memory...)
- You can consider the kNN like a debug tool control the data and the accuracy of the other classifiers (even neural classifiers)
- Last but not least: it tends to the Bayes optimal classifier....
 - the “god” classifier, the best classifier you can build even knowing exactly the probability distribution of your classes

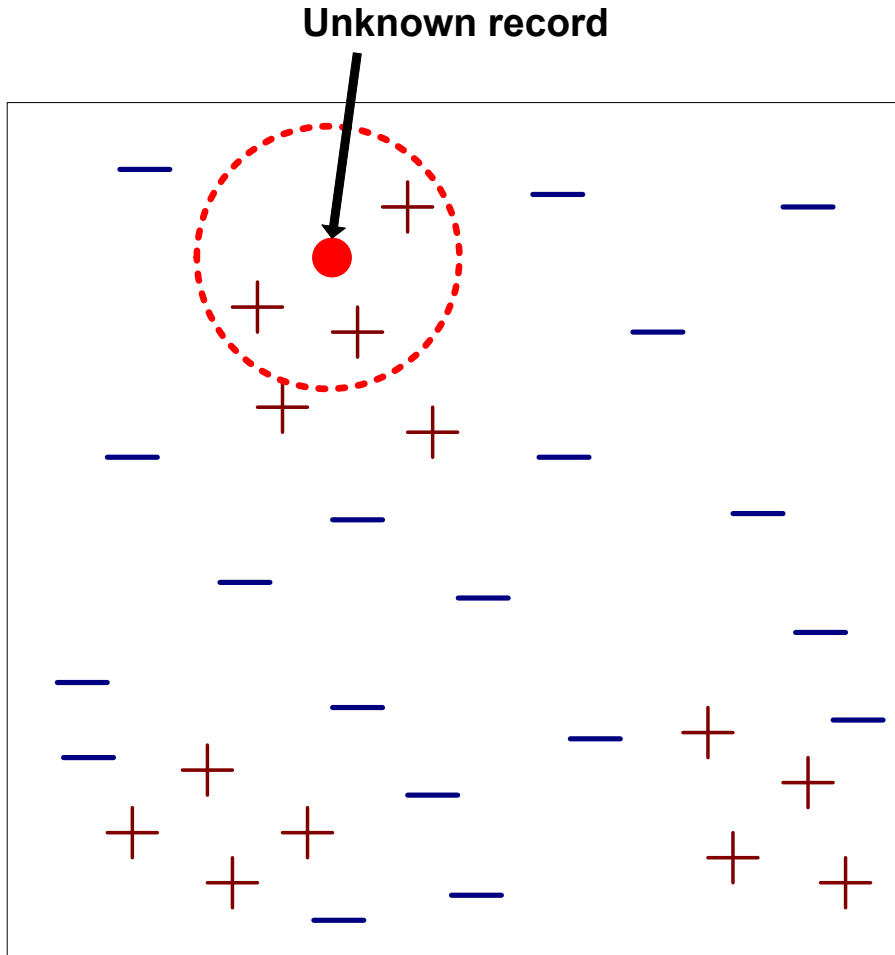
Nearest Neighbor Classifiers

Basic idea:

If it walks like a duck, quacks like a duck, then it's probably a duck

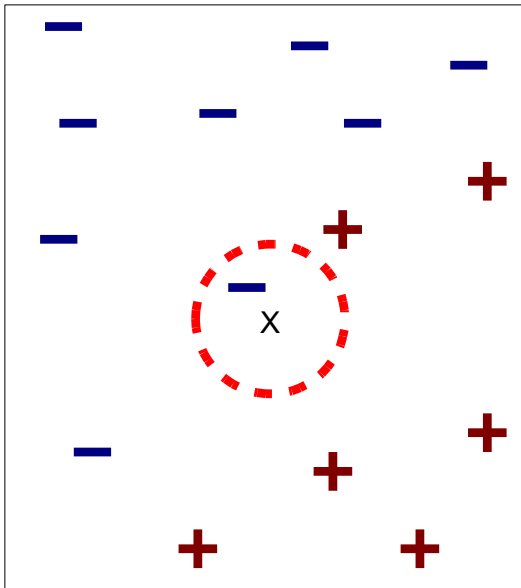


Nearest-Neighbor Classifiers

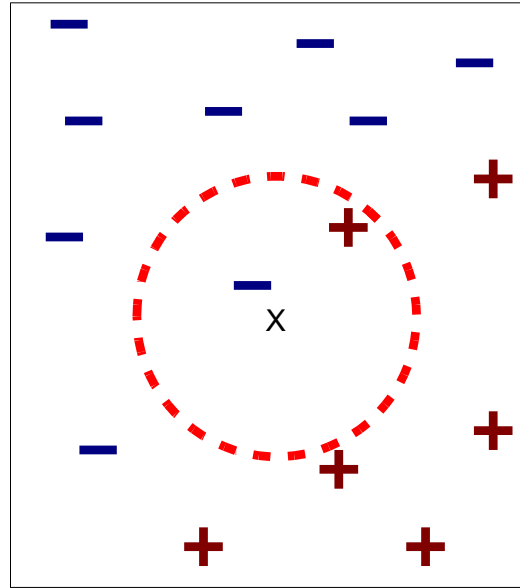


- Requires three things
 - The set of stored records
 - Distance Metric to compute distance between records
 - The value of k , the number of nearest neighbors to retrieve
- To classify an unknown record:
 - Compute distance to other training records
 - Identify k nearest neighbors
 - Use class labels of nearest neighbors to determine the class label of unknown record (e.g., by taking majority vote)

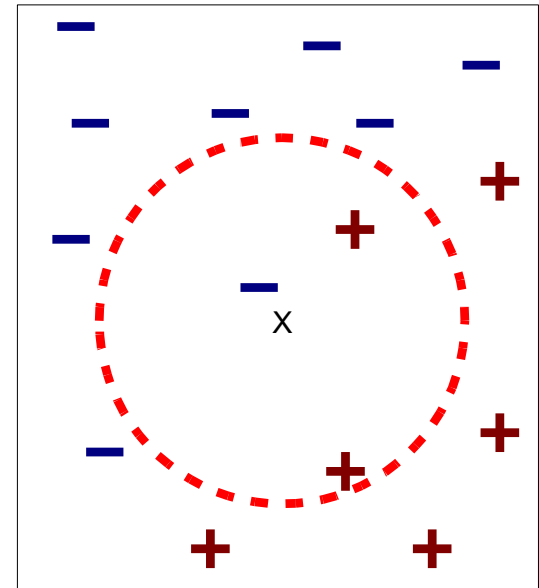
Definition of Nearest Neighbor



(a) 1-nearest neighbor



(b) 2-nearest neighbor

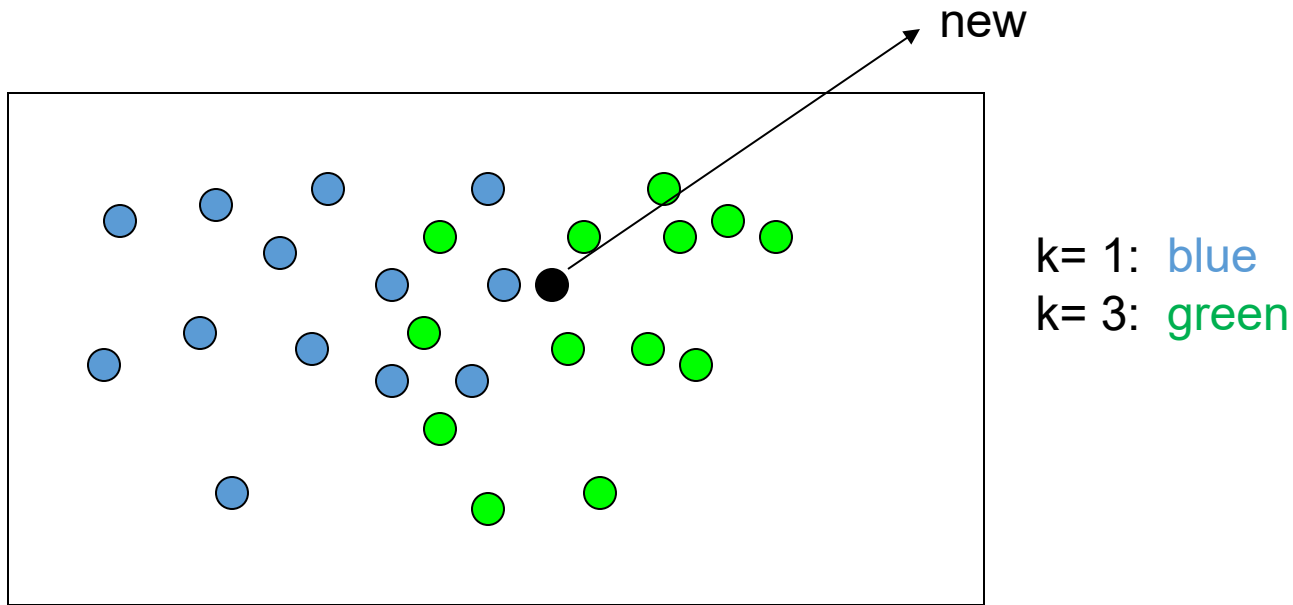


(c) 3-nearest neighbor

K-nearest neighbors of a record x are data points that have the k smallest distance to x

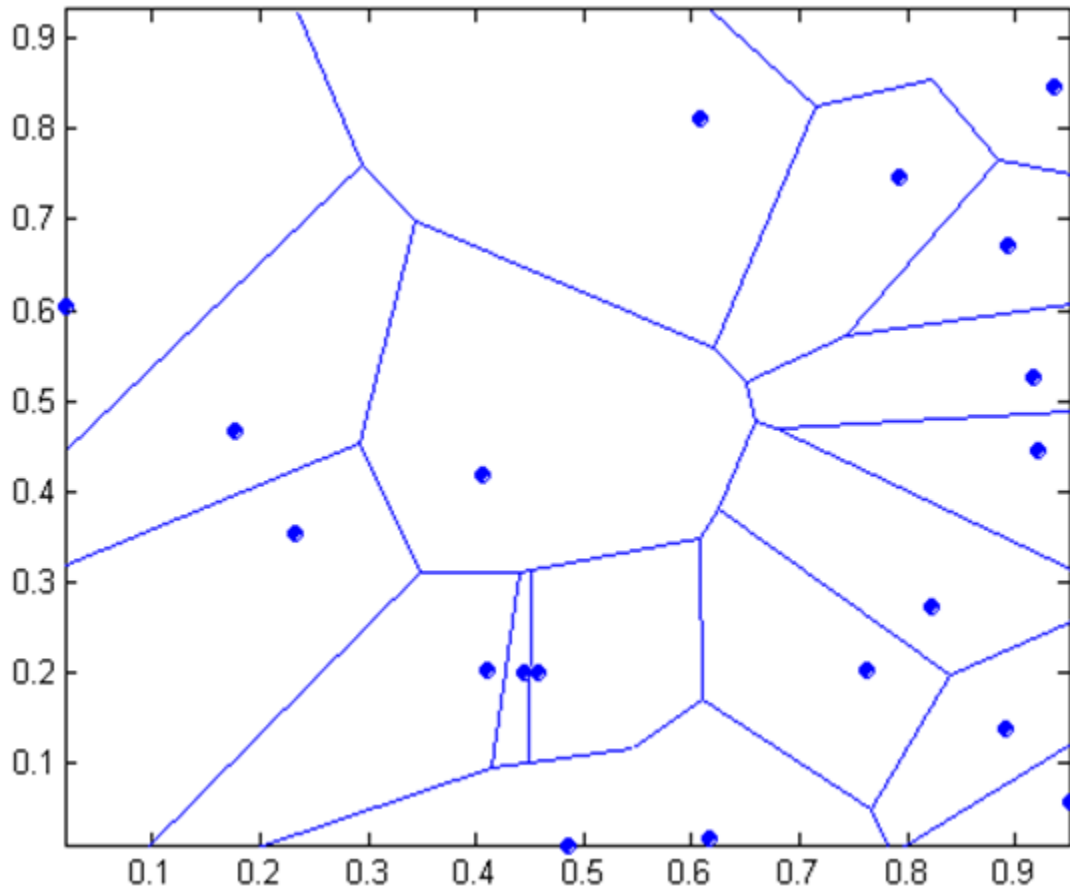
Nearest neighbor method

- Majority vote within the k nearest neighbors



The 1 nearest-neighbor decision boundaries

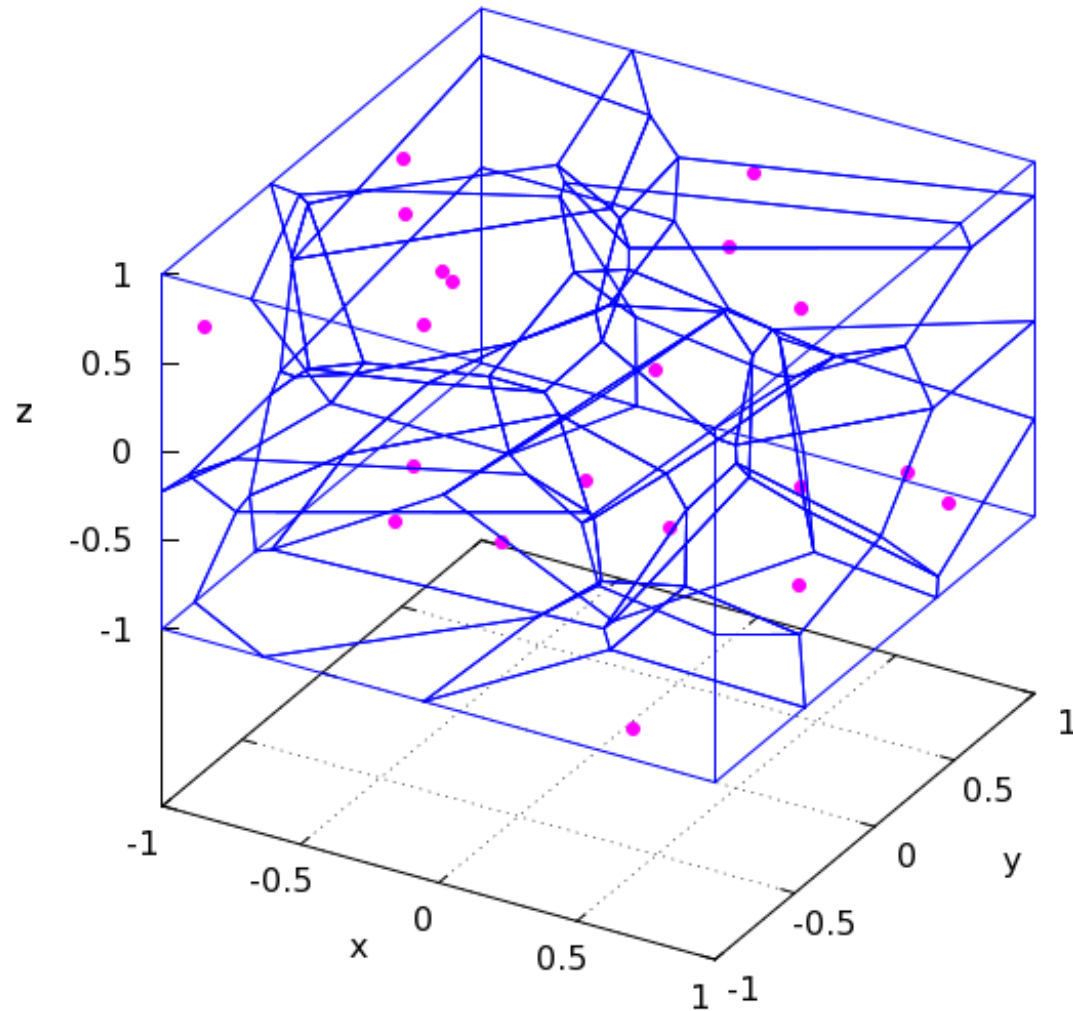
Voronoi Diagram



A partitioning of a plane into regions based on distance to points in a specific subset of the plane

The Voronoi diagram of a set of points is dual to its **Delaunay triangulation**.

NDimensional Voronois diagram

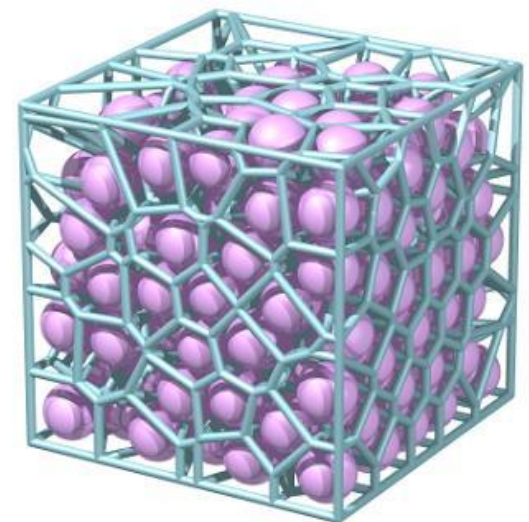
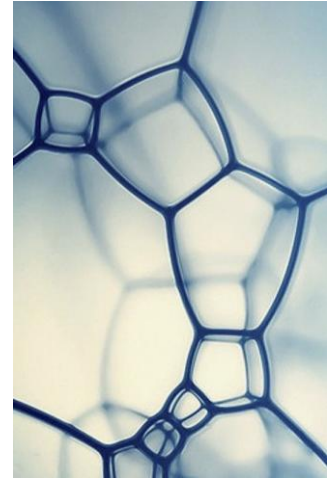


A 3D
example

Voronoi usage examples

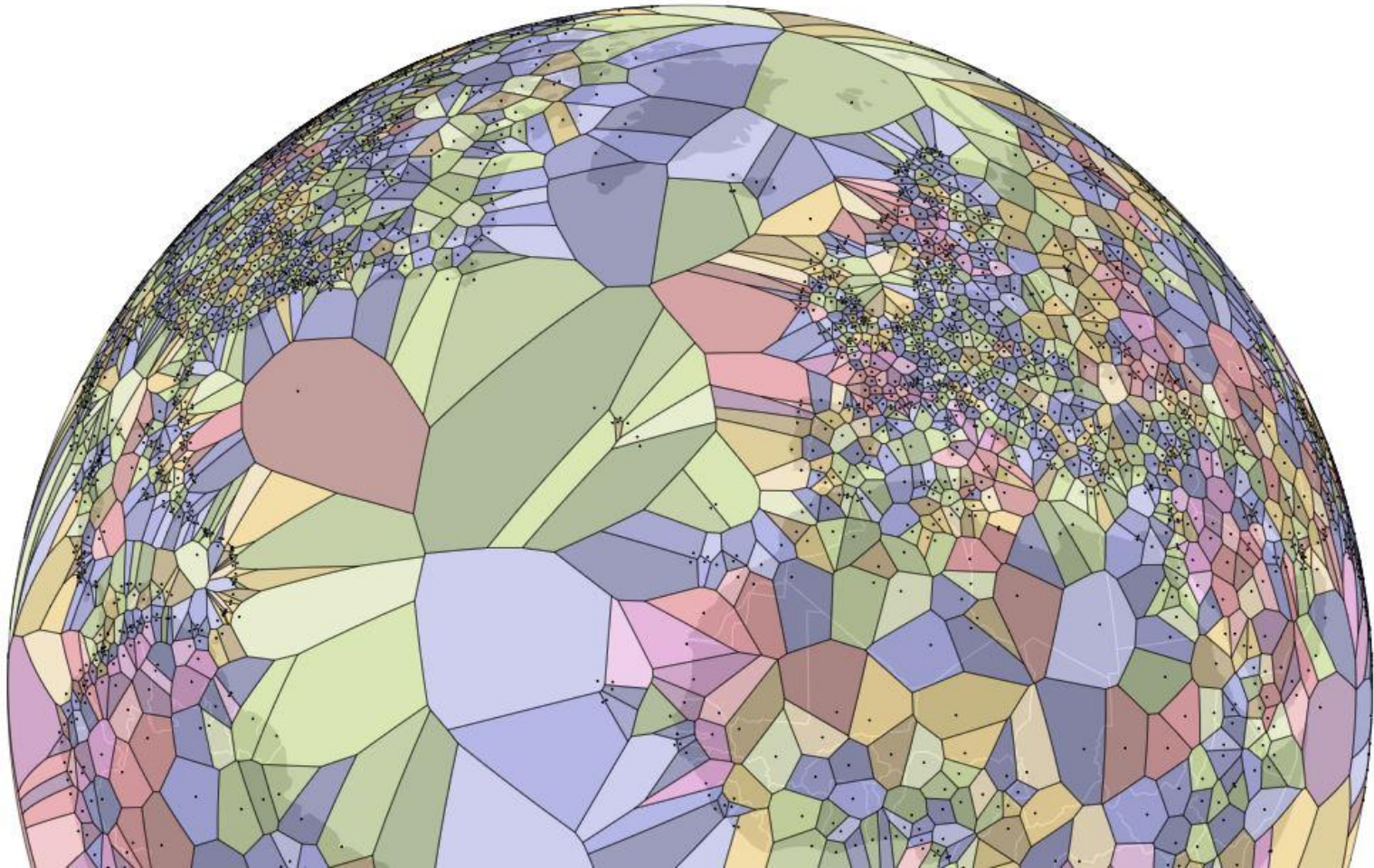


- Natural sciences and biology
 - Voronoi tessellation emerges by radial growth from seeds outward.
- Health
 - Correlate sources of infections in epidemics
- Engineering
 - Free volumes of polymers
- Geometry
- Informatics
- Civics and planning



Voronoi diagram usage

A voronoi diagram
of the world's
airports projected
onto an 3D globe
(Jason Davies)



Nearest Neighbor Classification

- Compute distance between two points:

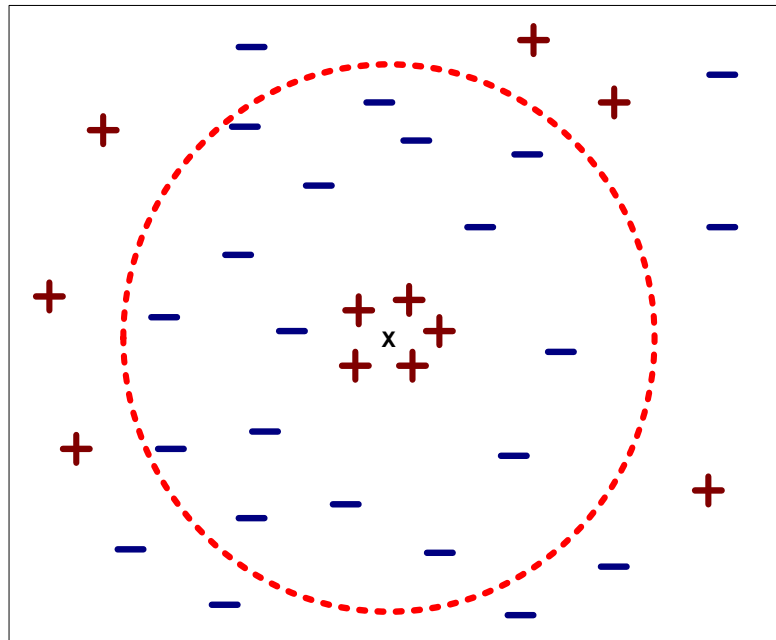
Example: Euclidean distance

$$d(p, q) = \sqrt{\sum_i (p_i - q_i)^2}$$

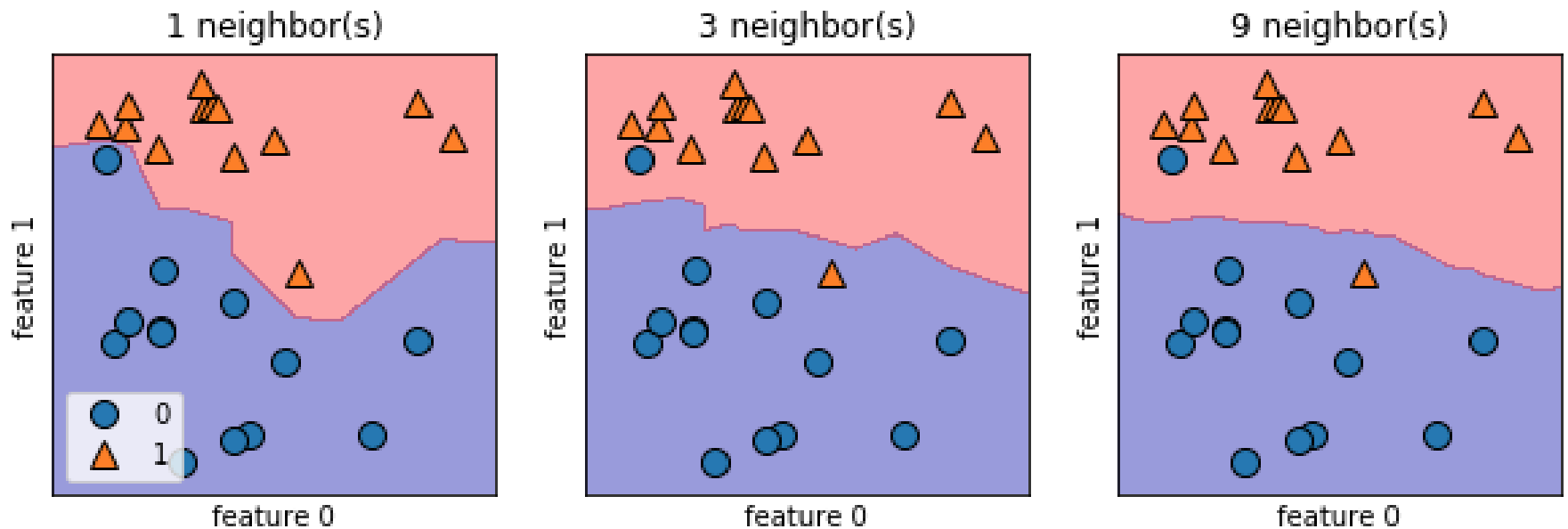
- Determine the class from nearest neighbor list
 - take the majority vote of class labels among the k-nearest neighbors
 - Alternative: weigh the vote according to distance
 - weight factor, $w = 1/d^2$

Design of the kNN: choosing the value of k

- If k is too small, sensitive to noise points
- If k is too large, neighborhood may include points from other classes

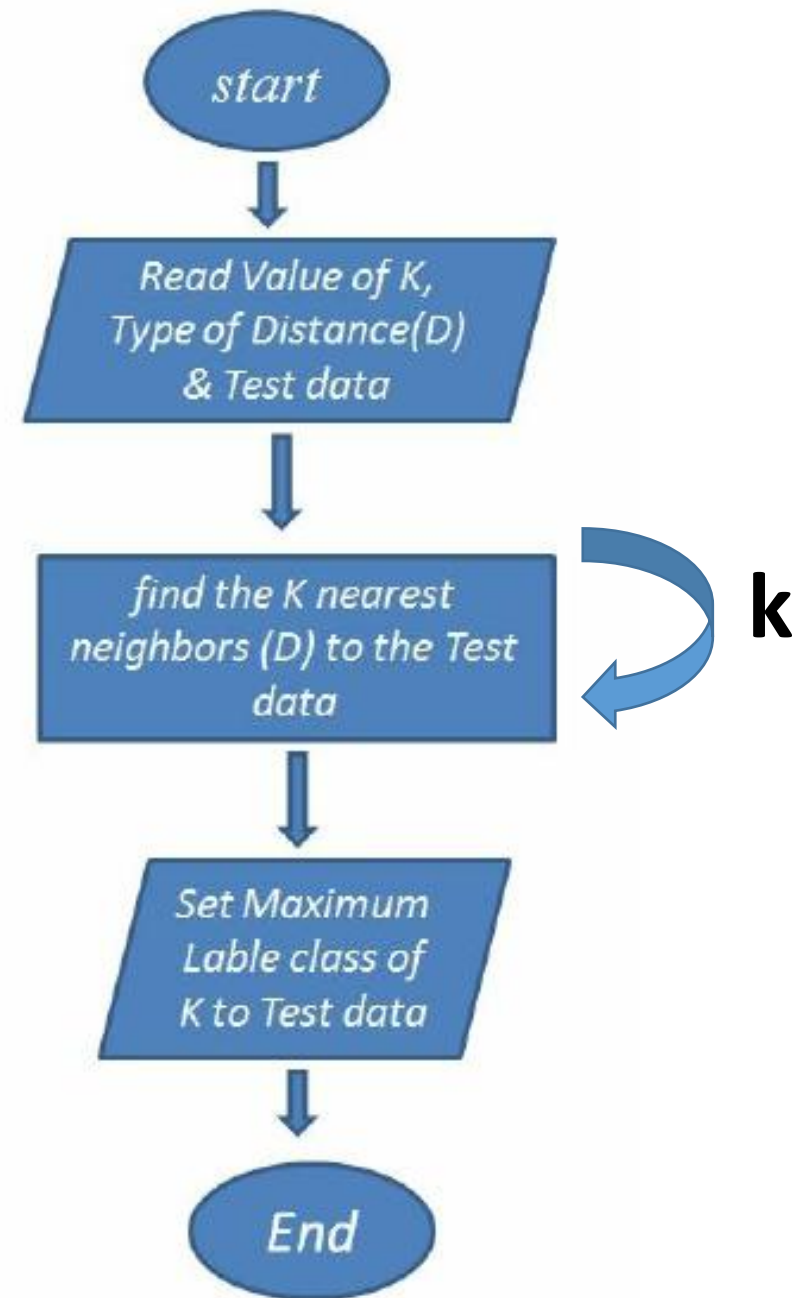
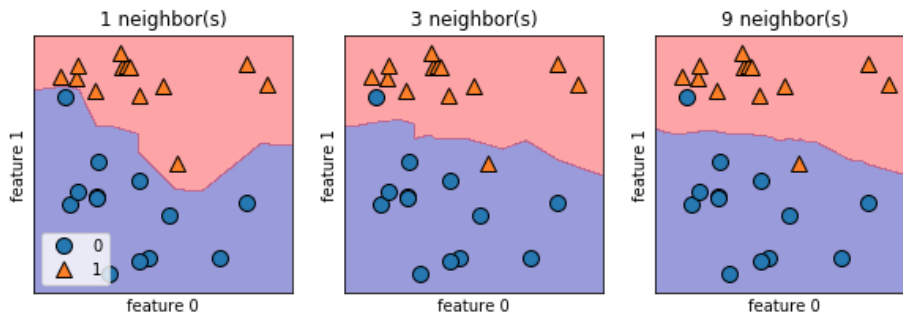


High $k \rightarrow$ regularization



... but more time is needed to process the stored data

Complexity $\leftrightarrow k$



Nearest Neighbor Classification...

• Scaling issues

- Attributes may have to be scaled to prevent distance measures from being dominated by one of the attributes
- Example:
 - height of a person may vary from 1.5m to 1.8m
 - weight of a person may vary from 90lb to 300lb
 - income of a person may vary from \$10K to \$1M

Discussion on the k-NN Algorithm

(+)

Robust to noisy data by averaging k-nearest neighbors

(-)

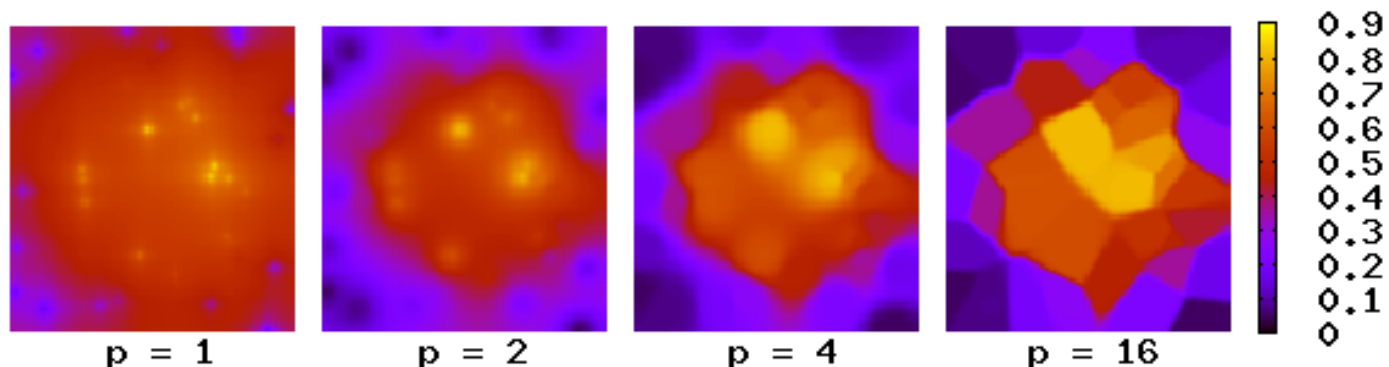
Curse of dimensionality: distance between neighbors could be dominated by irrelevant attributes.

To overcome it:

- axes stretch
- or elimination of the least relevant attributes (Feature selection)

Distance-Weighted Nearest Neighbor Algorithm

- Assign weights to the neighbors based on their 'distance' from the query point
 - Weight 'may' be inverse square of the distances
- ➔ All training points may influence a particular instance
 - Shepard's method



From smooth partitions → Voronoi

Practical issues when using kNN



SPEED



CURSE OF DIMENSIONALITY

Practical issues when using kNN:

Speed

- Time taken by kNN for N points of D dimensions
 - time to compute distances: $O(ND)$
 - time to find the k nearest neighbor
 - $O(kN)$: repeated minima
 - $O(N \log N)$: sorting
 - $O(N + k \log N)$: min heap
 - $O(N + k \log k)$: fast median
 - Total time is dominated by distance computation
- We can be faster if we are willing to sacrifice exactness

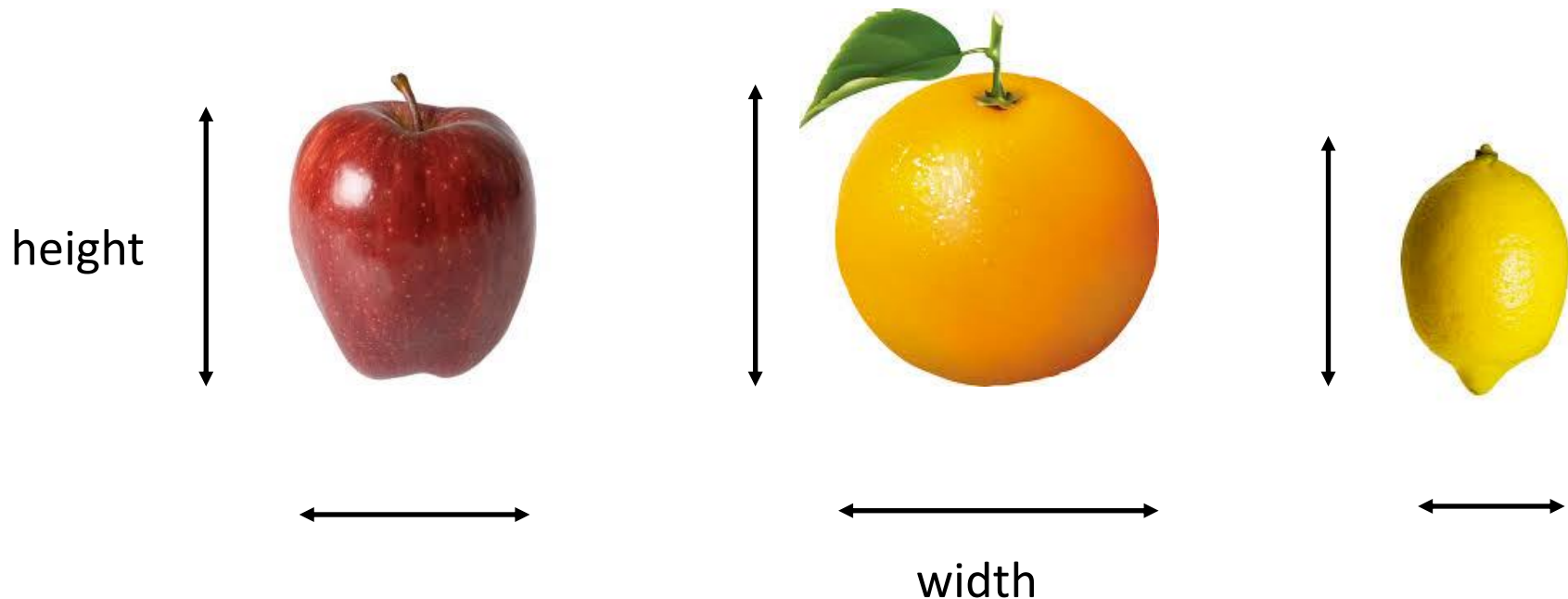


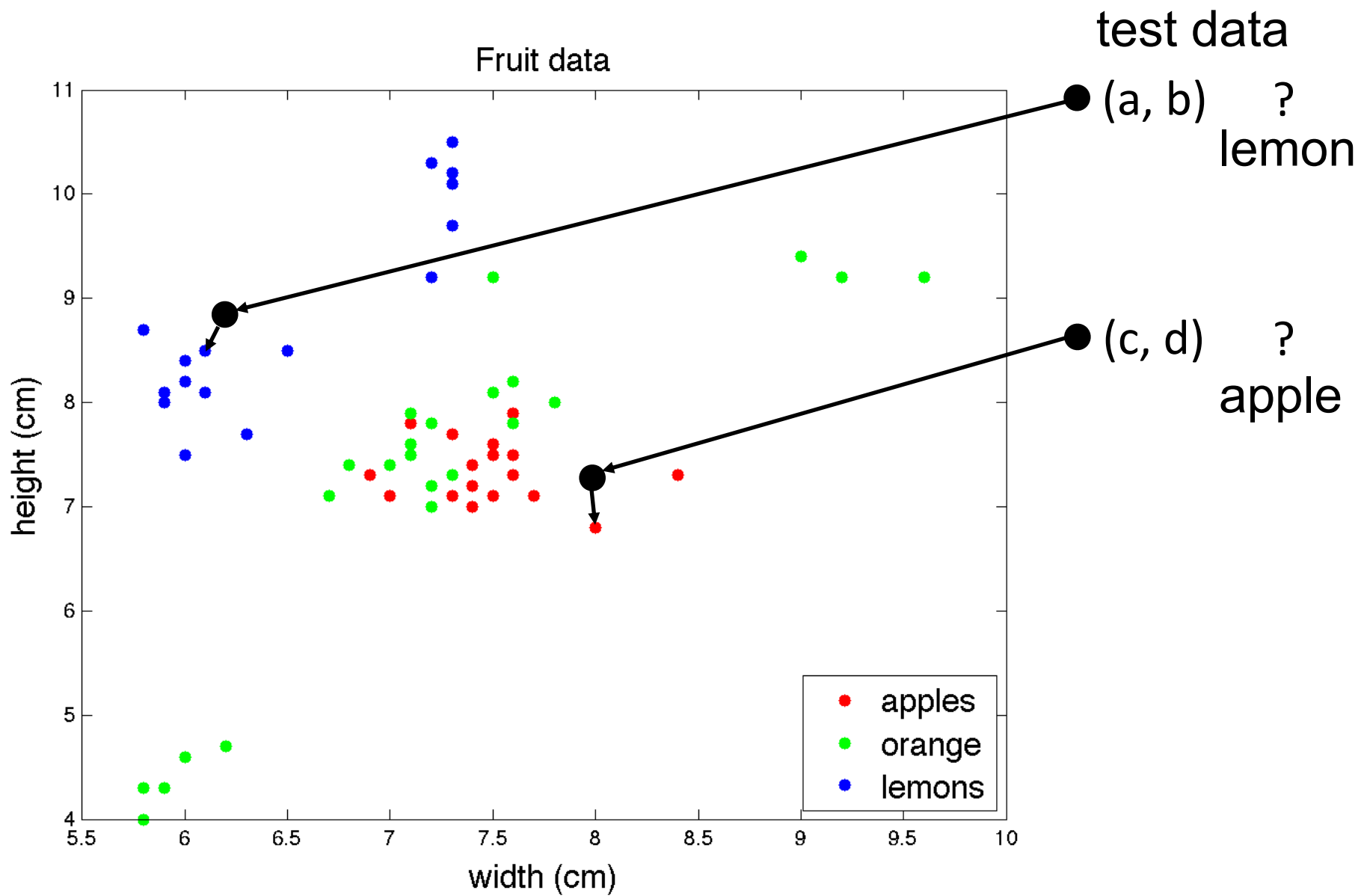


Example of kNN

Nearest neighbor classifier

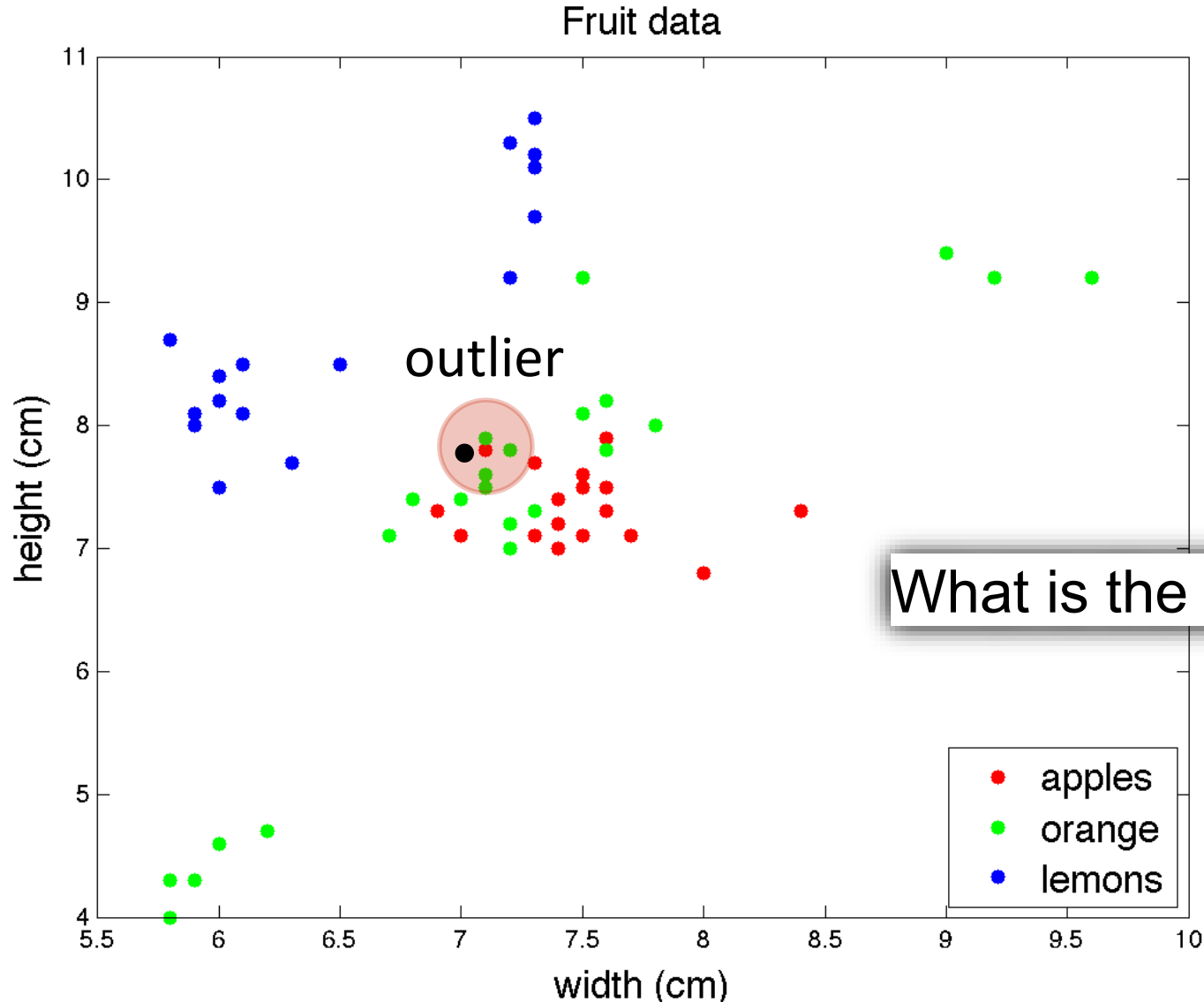
- ◆ Training data is in the form of $(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_n, y_n)$
- ◆ Fruit data:
 - label: {apples, oranges, lemons}
 - attributes: {width, height}





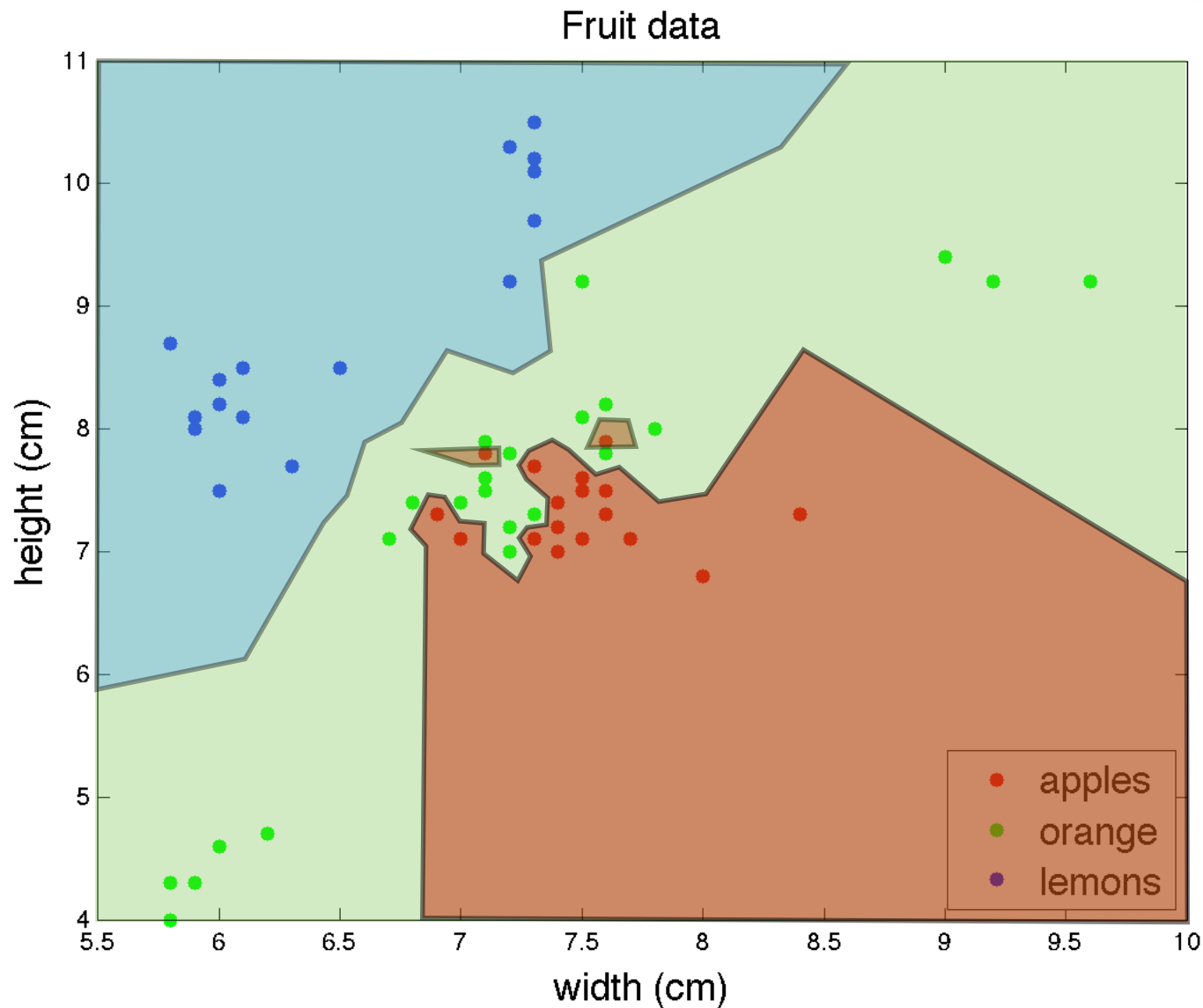
k-Nearest neighbor classifier

Take majority vote among the k nearest neighbors



Decision boundaries: 1NN

What is the effect of k?



Inductive **bias** of the kNN classifier

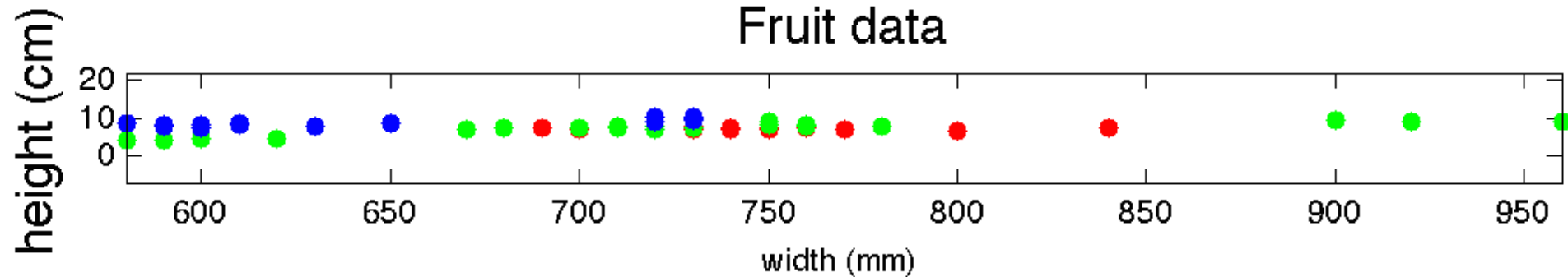
◆ Choice of features

- We are assuming that all features are equally important
- What happens if we scale one of the features by a factor of 100?

◆ Choice of distance function

- Euclidean, cosine similarity (angle), Gaussian, etc ...
- Should the coordinates be independent?

◆ Choice of k

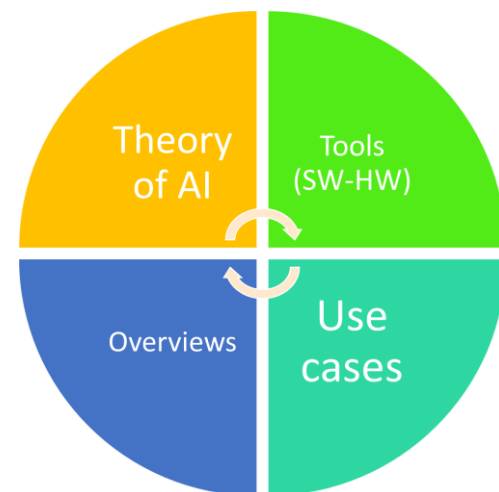




Toolboxes

Coding kNN

Matlab



kNN Coding in Matlab



laboratory_MATLAB_Knn.m

```
load fisheriris
```

```
X = meas;
```

```
Y = species;
```

```
% X is a numeric matrix that contains
```

```
% four petal measurements for 150 irises.
```

```
% Y is a cell array of character vectors
```

```
% that contains the corresponding iris species.
```

```
% just a simple plot
```

```
plotmatrix(X)
```

```
% Train a 5-nearest neighbor classifier.
```

```
% Standardize the noncategorical predictor data
```

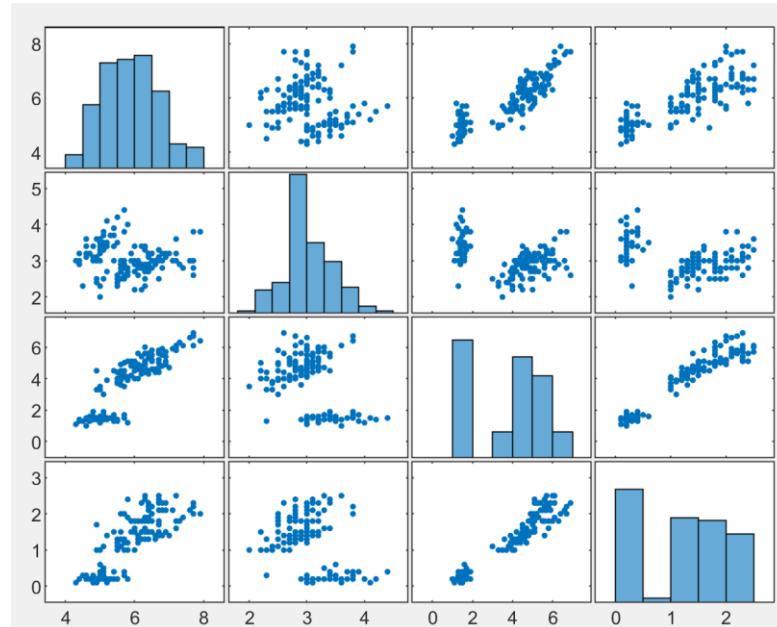
```
% --> see lesson about encoding the outputs
```

```
kNN_model = fitcknn(X,Y,'NumNeighbors',5,'Standardize',1)
```

```
% Let's try to input a single vector
```

```
x = X(1,:)
```

```
label = predict(kNN_model,x)
```



```
kNN_model =
```

[ClassificationKNN](#)

```
ResponseName: 'Y'
```

```
CategoricalPredictors: []
```

```
ClassNames: {'setosa' 'versicolor' 'virginica'}
```

```
ScoreTransform: 'none'
```

```
NumObservations: 150
```

```
Distance: 'euclidean'
```

```
NumNeighbors: 5
```

```
x =
```

```
5.1000 3.5000 1.4000 0.2000
```

```
label =
```

```
1x1 cell array
```

```
{'setosa'}
```

$$z_j = \frac{x_j - \mu_j}{\sigma_j}$$

kNN Coding in Matlab

% Do a cross-validation test

% The function will create SUBCLASSIFIERS to do a correct CrossValidation

```
cvmdl_results = crossval(kNN_model, 'KFold', 10)
```

```
kfoldLoss(cvmdl_results)
```

ans =

0.0533

```
cvmdl_results =
```

```
classreg.learning.partition.ClassificationPartitionedModel
```

```
CrossValidatedModel: 'KNN'
```

```
PredictorNames: {'x1' 'x2' 'x3' 'x4'}
```

```
ResponseName: 'Y'
```

```
NumObservations: 150
```

```
KFold: 10
```

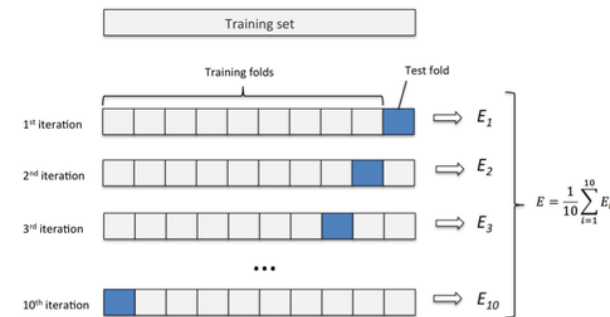
```
Partition: [1x1 cvpartition]
```

```
ClassNames: {'setosa' 'versicolor' 'virginica'}
```

```
ScoreTransform: 'none'
```

What happens if we change the validation folders?

From k=10 to 3



% Let's do a cross-validation test with different folder from 10 to 3?

% K = 10

```
fprintf('KFOLD Validation with K = %d --> Error = %f \n', 10, kfoldLoss(cvmdl_results));
```

% K = 9...3

```
for K = [9:-1:3]
```

```
    cvmdl_results = crossval(kNN_model, 'KFold', K);
```

```
    fprintf('KFOLD Validation with K = %d --> Error = %f \n', K, kfoldLoss(cvmdl_results));
```

```
end
```

```
KFOLD Validation with K = 10 --> Error = 0.053333
KFOLD Validation with K = 9 --> Error = 0.046667
KFOLD Validation with K = 8 --> Error = 0.053333
KFOLD Validation with K = 7 --> Error = 0.046667
KFOLD Validation with K = 6 --> Error = 0.053333
KFOLD Validation with K = 5 --> Error = 0.060000
KFOLD Validation with K = 4 --> Error = 0.046667
KFOLD Validation with K = 3 --> Error = 0.046667
```

Less information in the training (in general) → accuracy is getting worse, but noise is masking this effect here

Warning:

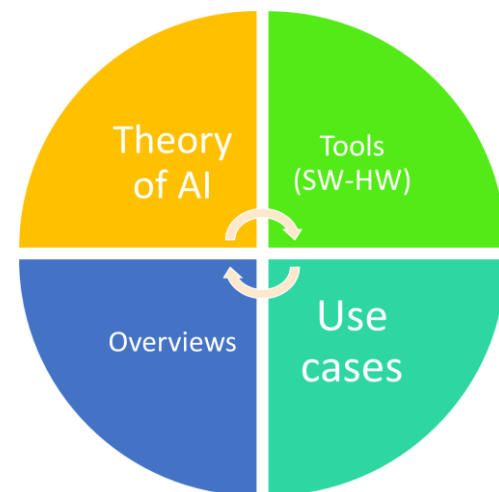
That is the **K of the CV**
Not the k = 5 of the kNN!



Toolboxes

Coding Classifiers

Till now in Matlab e Python



```
x_new = np.array([[1.5, 2.7]])
```

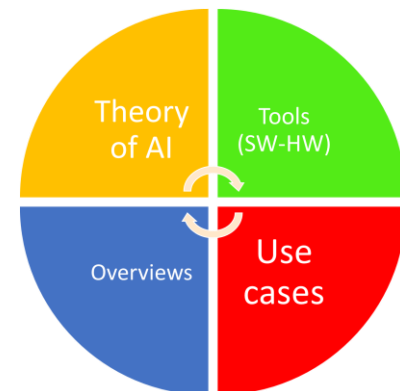
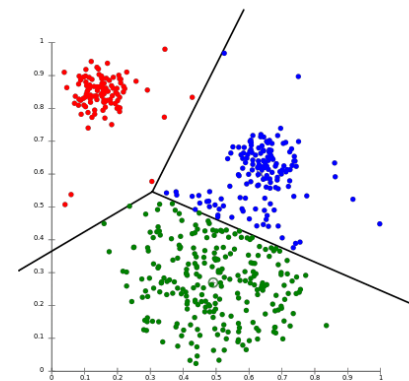
Model	MATLAB	Python (scikit-learn)
Polynomial Fit (degree 3) <i>Univariate (1 variable)</i>	<pre>p = polyfit(x, y, 3); y_new = polyval(p, x_new);</pre>	<pre>p = numpy.polyfit(x, y, 3) y_new = numpy.polyval(p, x_new)</pre>
Polynomial Fit (degree 3) <i>Multivariate (2 variables)</i>	<pre>mdl = fitlm(X, y, 'poly3'); y_new = predict(mdl, x_new);</pre>	<pre>from sklearn.preprocessing import PolynomialFeatures from sklearn.linear_model import LinearRegression poly = PolynomialFeatures(degree=3) X_poly = poly.fit_transform(X) mdl = LinearRegression().fit(X_poly, y) x_new_poly = poly.transform([x_new]) y_new = mdl.predict(x_new_poly)</pre>
Linear Classifier	<pre>mdl = fitclinear(X, y); y_new = predict(mdl, x_new);</pre>	<pre>from sklearn.linear_model import LogisticRegression mdl = LogisticRegression().fit(X, y) y_new = mdl.predict([x_new])</pre>
kNN Classifier	<pre>mdl = fitcknn(X, y, 'NumNeighbors', 5); y_new = predict(mdl, x_new);</pre>	<pre>from sklearn.neighbors import KNeighborsClassifier mdl = KNeighborsClassifier(n_neighbors=5).fit(X, y) y_new = mdl.predict([x_new])</pre>
Decision Tree	<pre>mdl = fitctree(X, y); y_new = predict(mdl, x_new);</pre>	<pre>from sklearn.tree import DecisionTreeClassifier mdl = DecisionTreeClassifier().fit(X, y) y_new = mdl.predict([x_new])</pre>
Naive Bayes Classifier	<pre>mdl = fitcnb(X, y); y_new = predict(mdl, x_new);</pre>	<pre>from sklearn.naive_bayes import GaussianNB mdl = GaussianNB().fit(X, y) y_new = mdl.predict([x_new])</pre>



THEORY

Clustering: k-Means

An unsupervised method

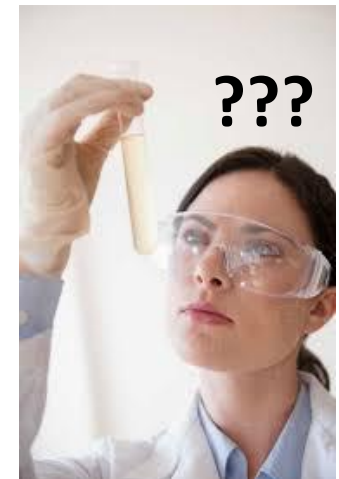
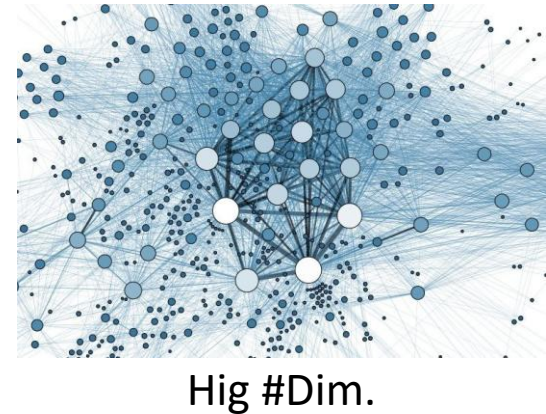
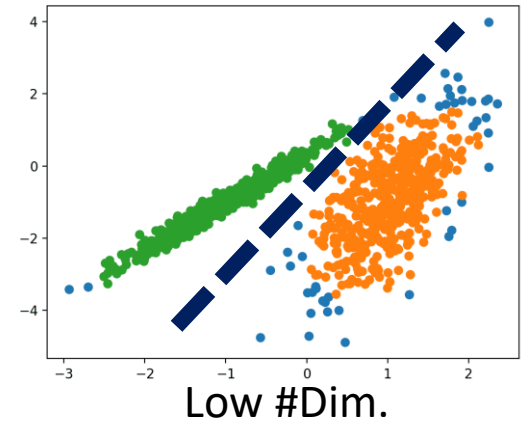


Imagine these assignments

- You receive an unsupervised dataset, for example about
 - **Customers data** and you are asked to try to define new type of customers
 - **Signals and vibrations** coming from an engine and you are asked to try to define if there are faulty states in the system
 - **Server logs** trying to understand if there are critical times and attacks on it
- What can be a good technique to face this problem? **Clustering is a very good answer!**

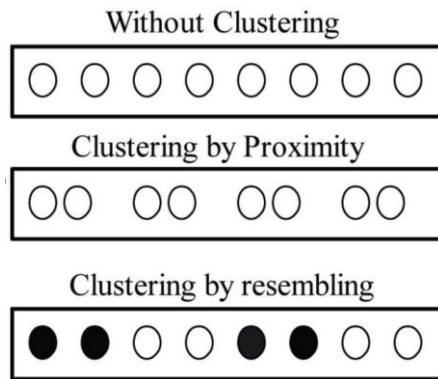
Clustering rationale

- In applications vary rarely you can identify the clusters in data by hard coding since
 - The dimensionality of the problem is very high
 - You are not (yet) an expert in the field (for example a gene expression in medicine, a complex parameter in finance, etc..)
 - The quantity of data is huge
 - ...
- You need an automated method!

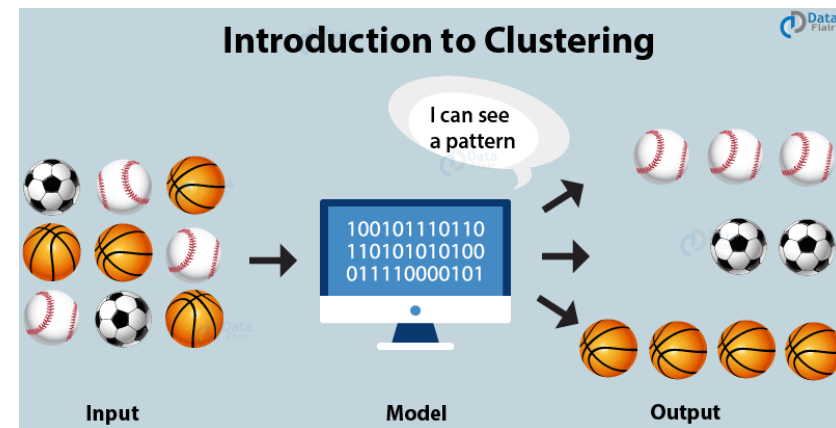
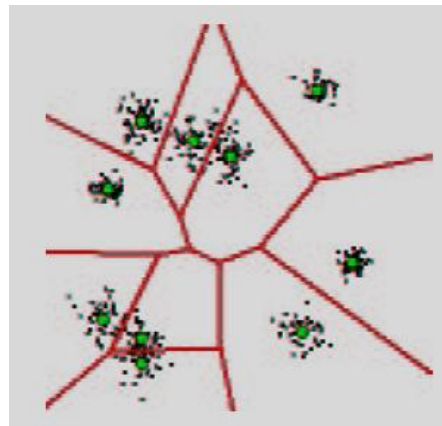


Clustering

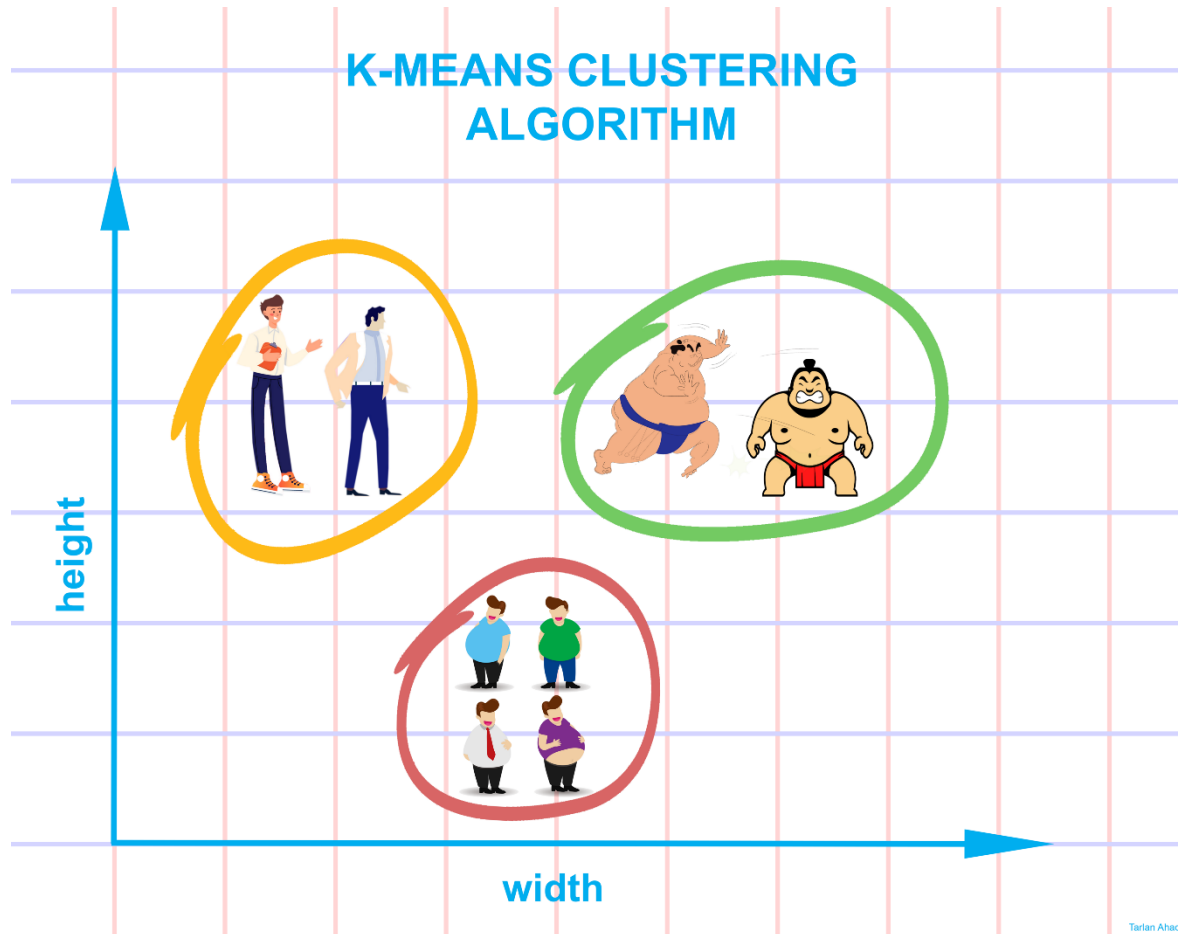
Cluster analysis or clustering is the task of **grouping a set of objects** in such a way that objects in the same group (called a cluster) are more similar (position, features, shapes, ...) to each other than to those in other groups (clusters)



Human Gestalt



Clustering



Clustering



Making groups of similar features

Kid/Adult?
Male/Female?
Happy/Neutral/Sad?
Long hair/bold?
Beard/Shaved?
Sunglasses/glasses/none?

...

We need to extract groups
from an unsupervised
dataset of features

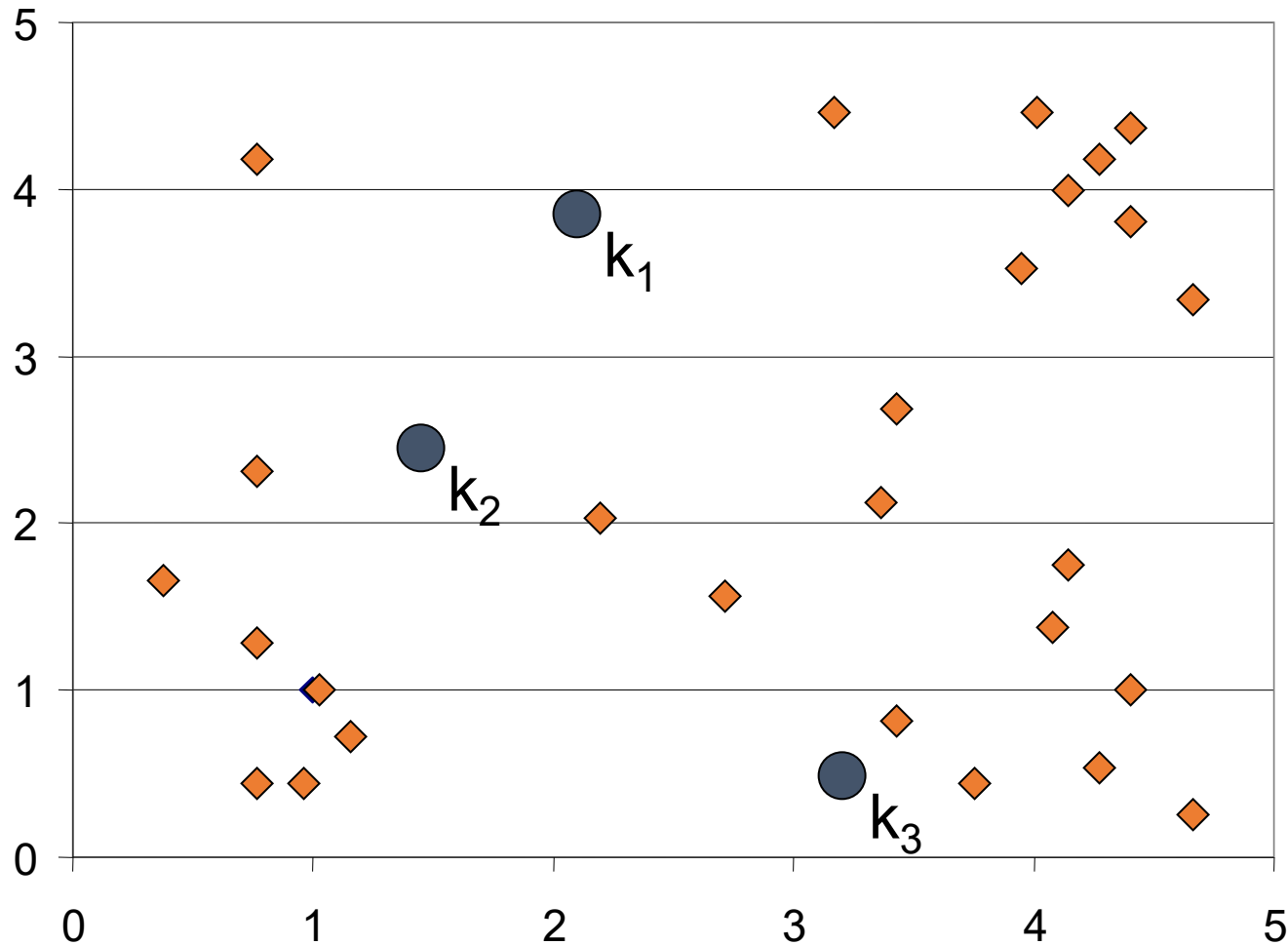
Why do we cluster?

- **Summarizing data**
 - Look at large amounts of data
 - Patch-based compression or denoising
 - looking the centroids with respect the whole dataset
 - Represent a large continuous vector with the cluster number
 - data points are so dense you can see the transitions
- **Counting**
 - Histograms of texture, color, SIFT vectors
- **Segmentation**
 - Separate the image into different regions
- **Prediction**
 - Images in the same cluster may have the same labels

k-means Clustering: Step 1

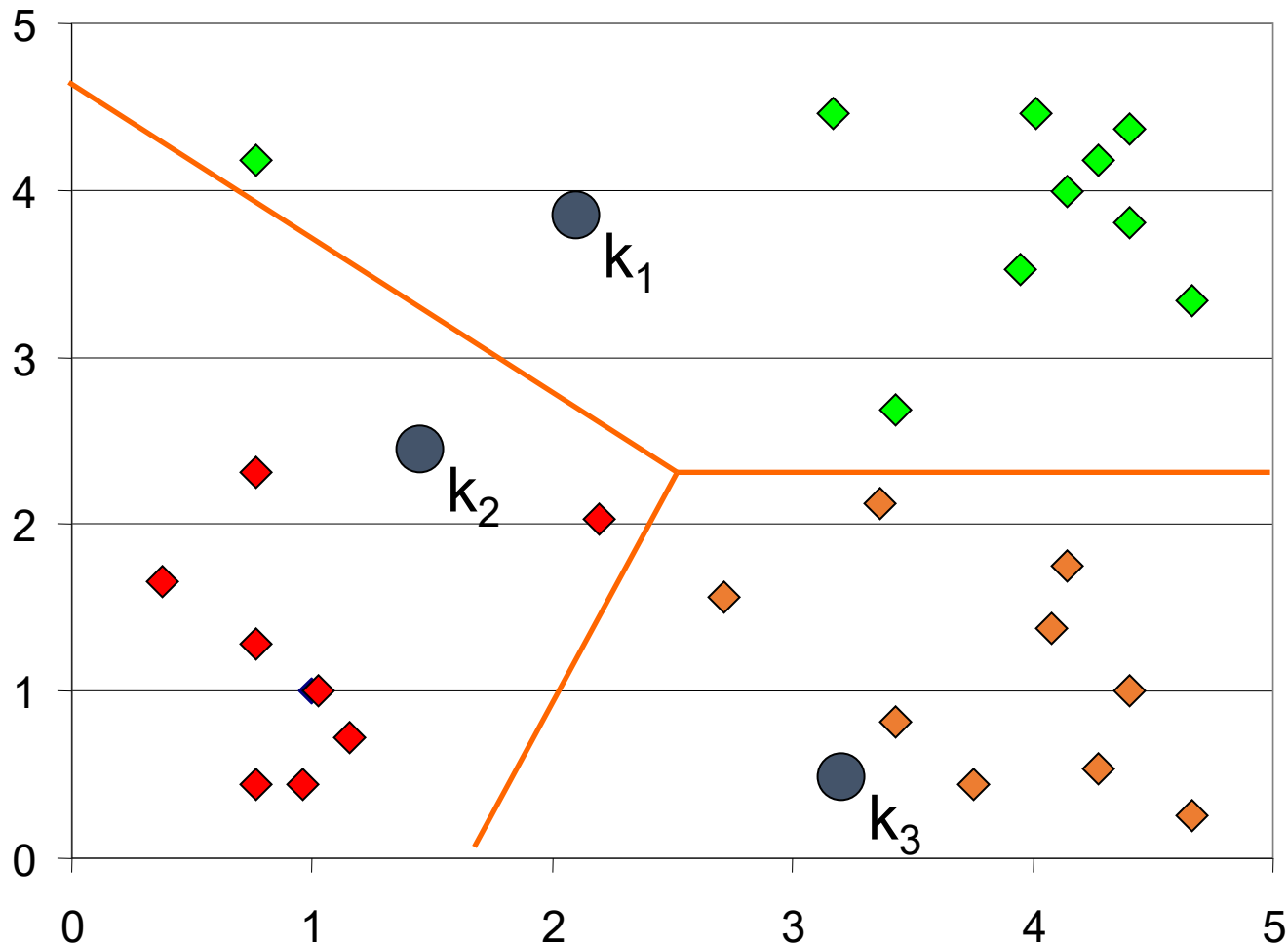
Random initialization

Algorithm: k-means, Distance Metric: Euclidean Distance



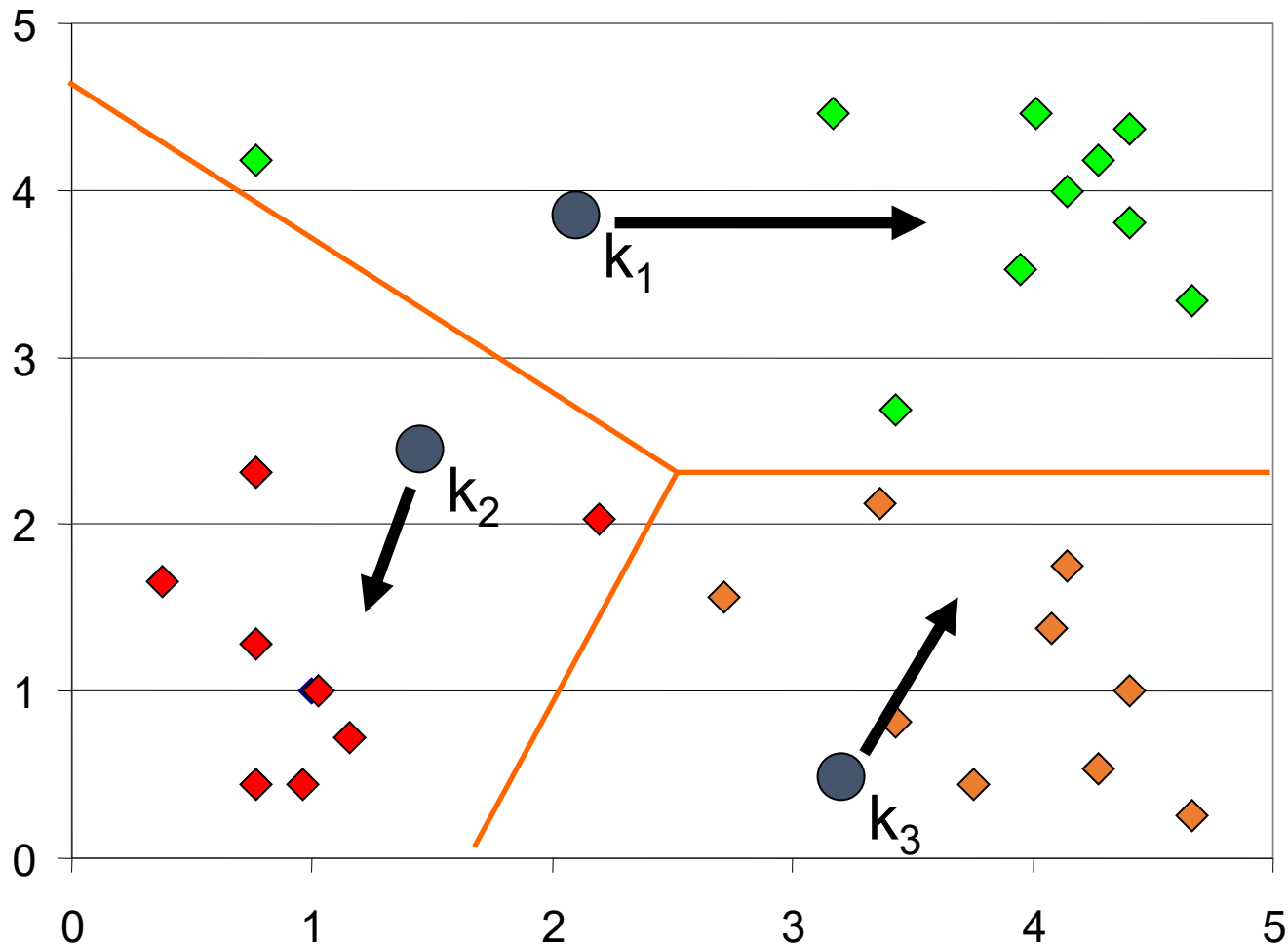
k-means Clustering: Step 2

Algorithm: k-means, Distance Metric: Euclidean Distance



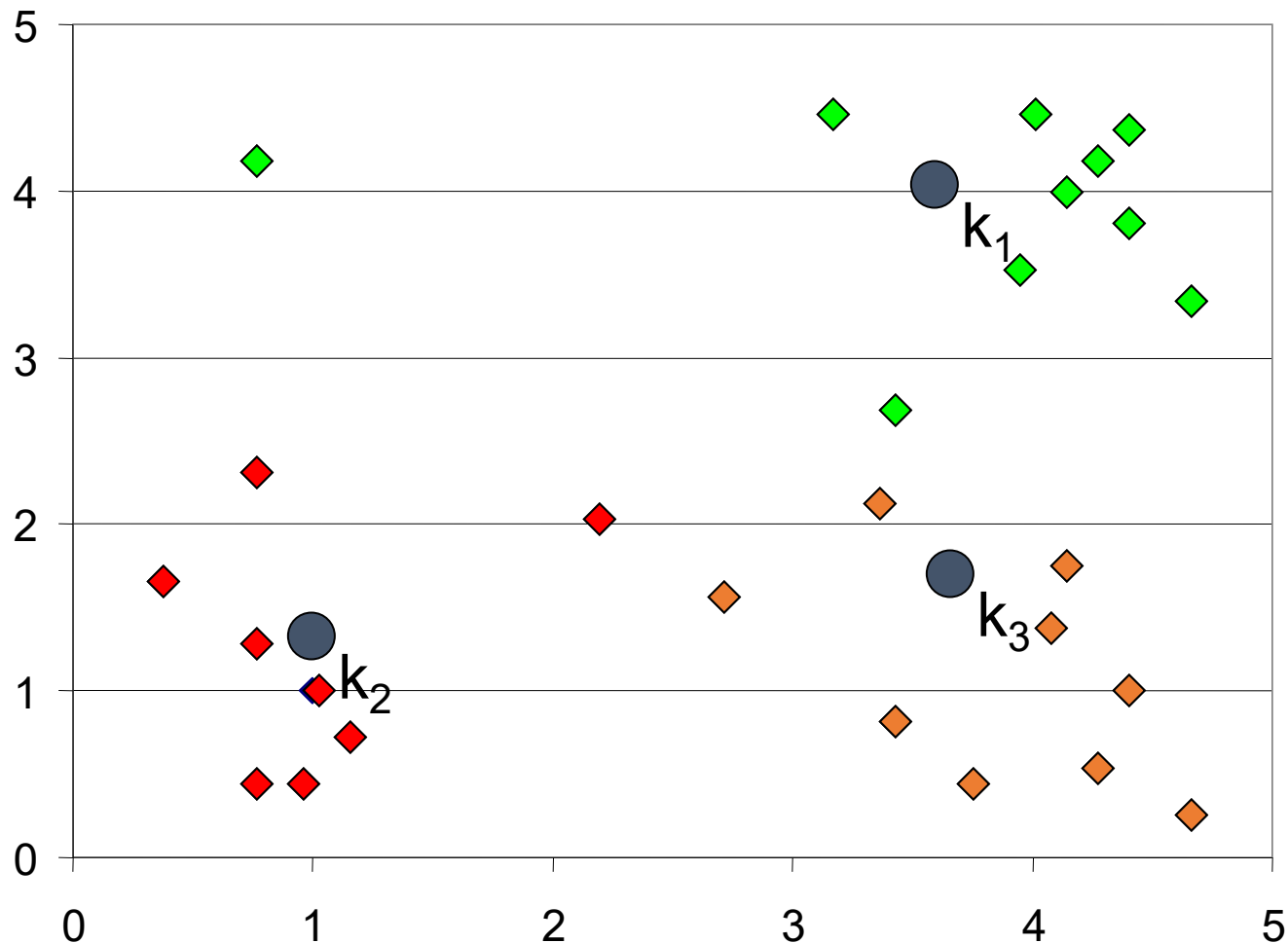
k-means Clustering: Step 2

Algorithm: k-means, Distance Metric: Euclidean Distance



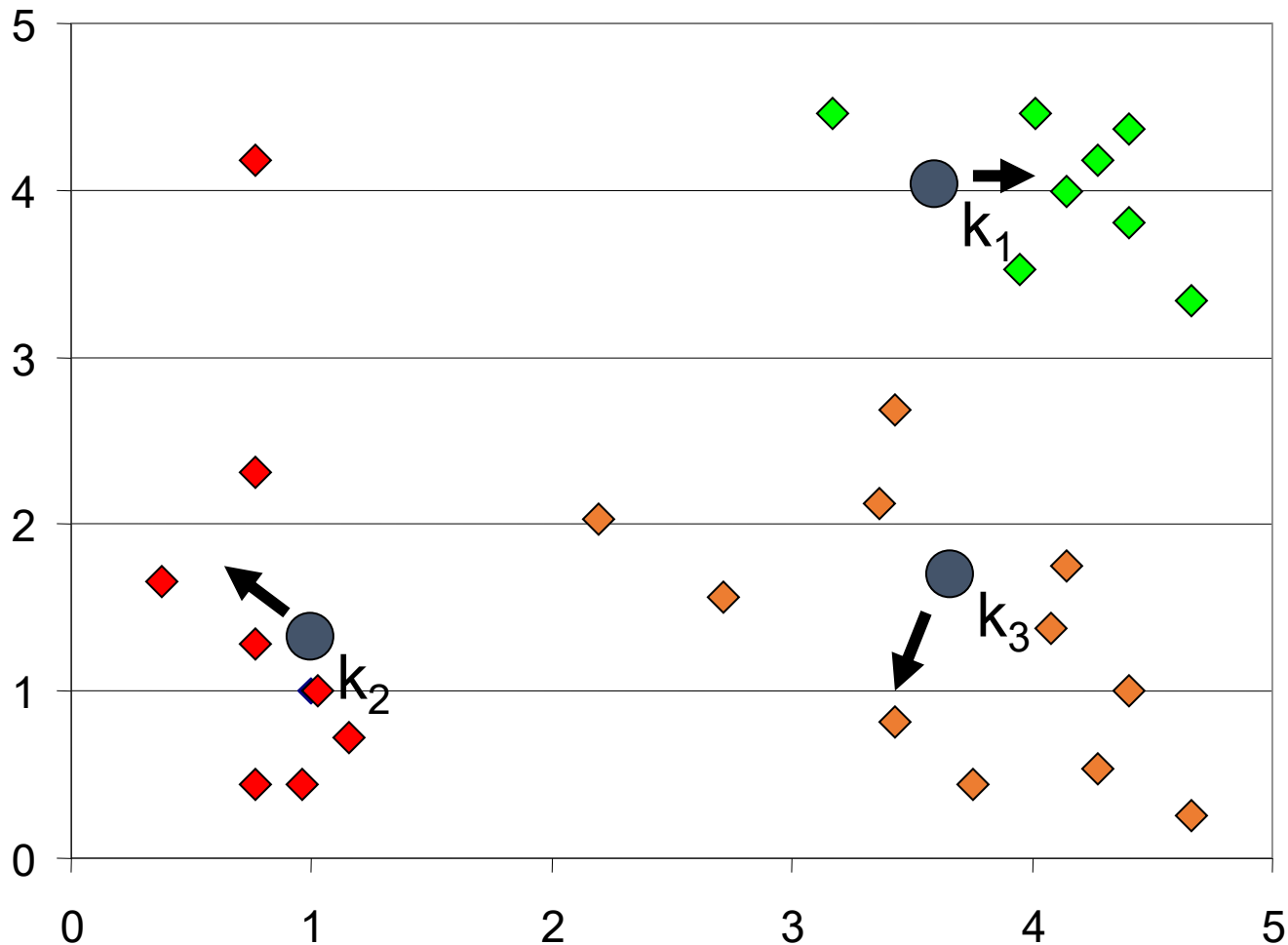
k-means Clustering: Step 3

Algorithm: k-means, Distance Metric: Euclidean Distance



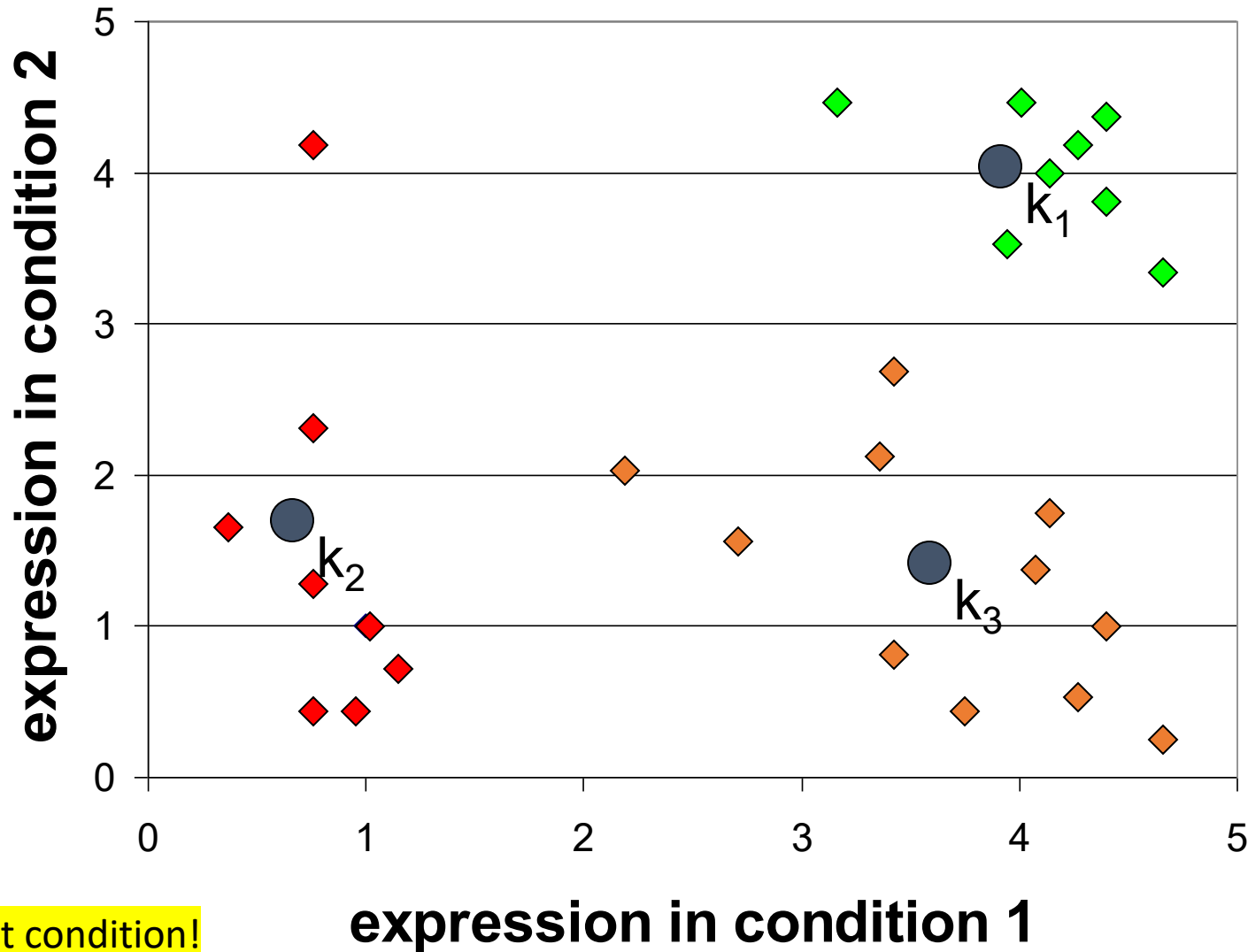
k-means Clustering: Step 4

Algorithm: k-means, Distance Metric: Euclidean Distance



k-means Clustering: Step 5

Algorithm: k-means, Distance Metric: Euclidean Distance

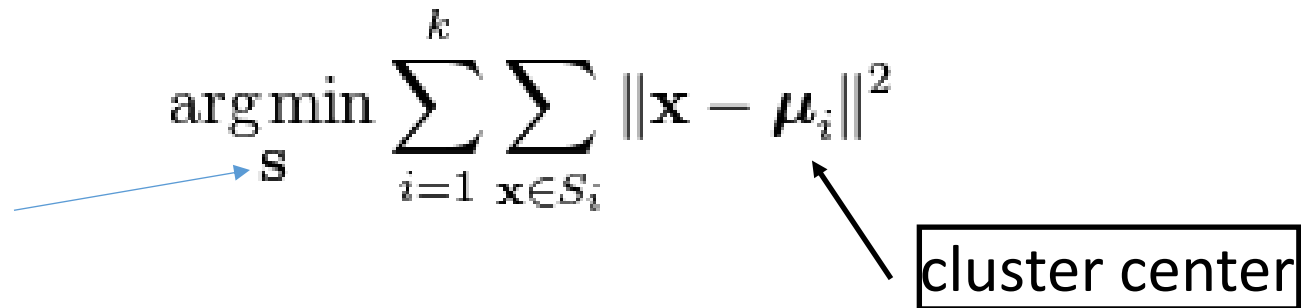


That's the exit condition!
Centers are not moving any more

Clustering using k-means

- Given $(x_1, x_2, \dots, x_n)$ observations, k-means clustering aims to partition the n observations into k ($\leq n$) sets $\mathbf{S} = \{S_1, S_2, \dots, S_k\}$ so as to minimize the within-cluster sum of squares.
- In other words, its objective is to find:

$$\underset{\mathbf{S}}{\operatorname{argmin}} \sum_{i=1}^k \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2$$



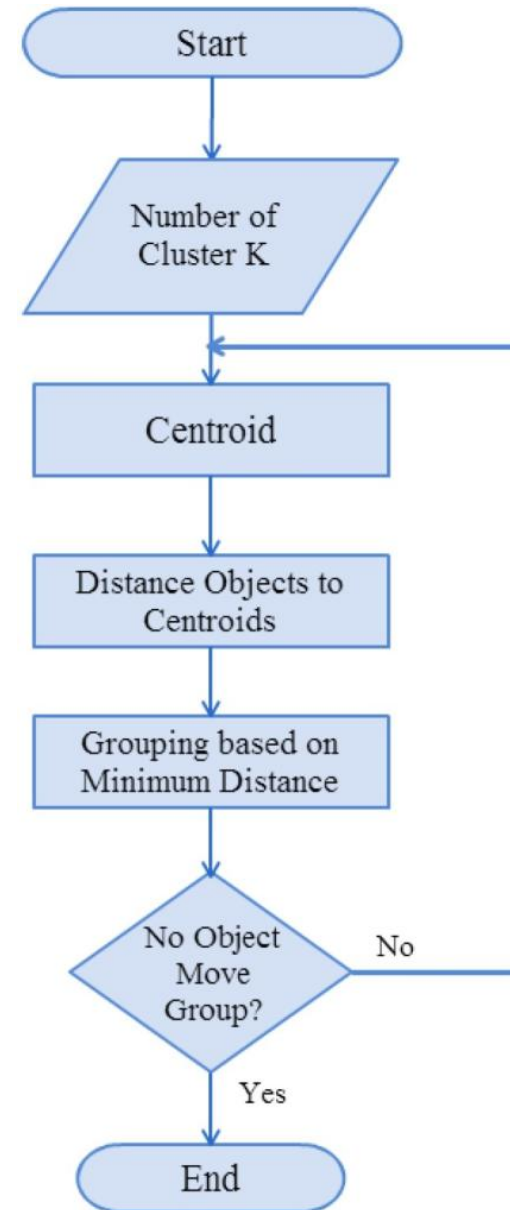
Easy to compute $\boldsymbol{\mu}$ given $\mathbf{S}$ and vice versa.

$k (\leq n)$ sets $S = \{S_1, S_2, \dots, S_k\}$

$$\arg \min_{\mathbf{S}} \sum_{i=1}^k \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \mu_i\|^2$$

cluster center

↑

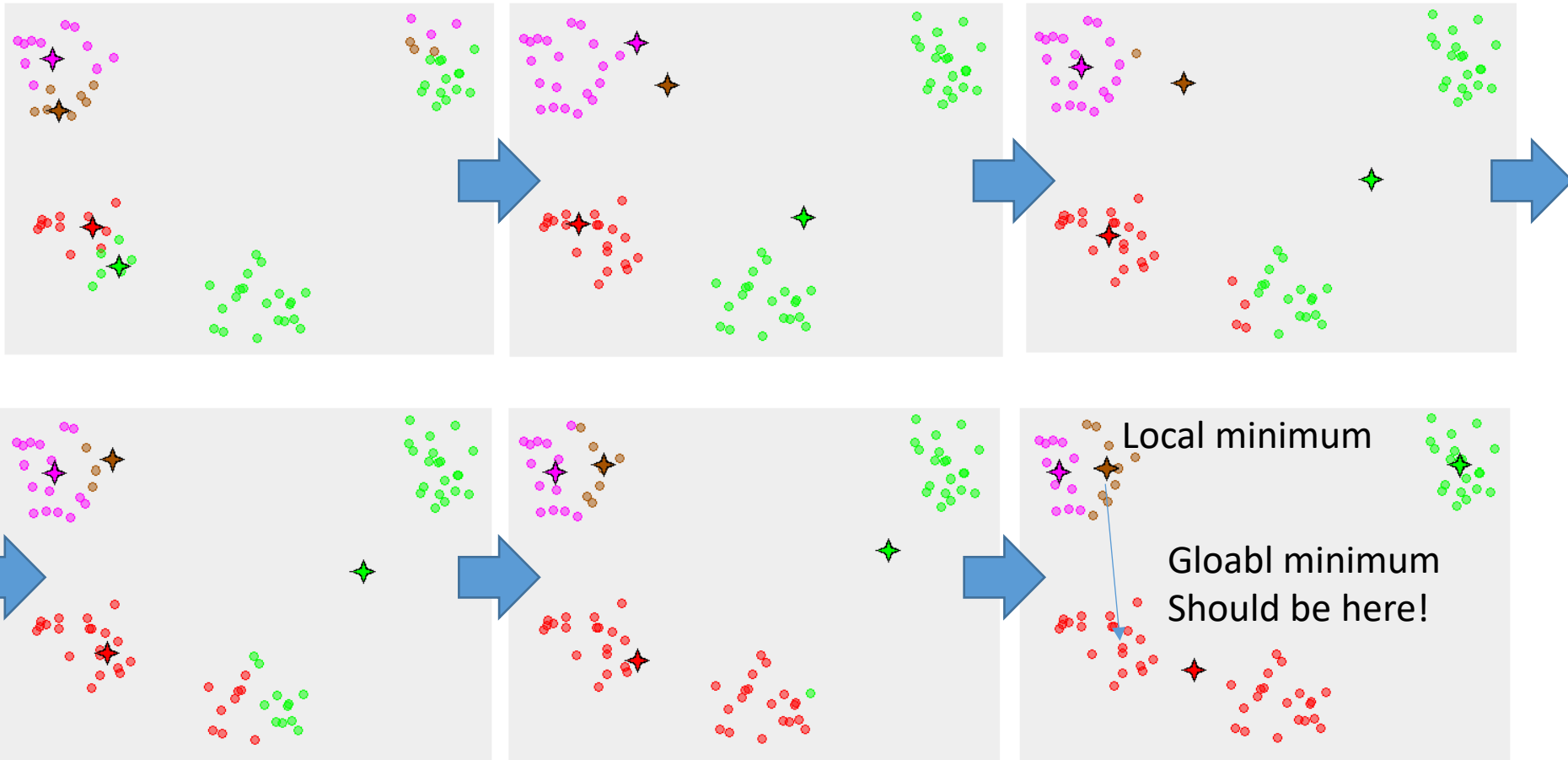


S

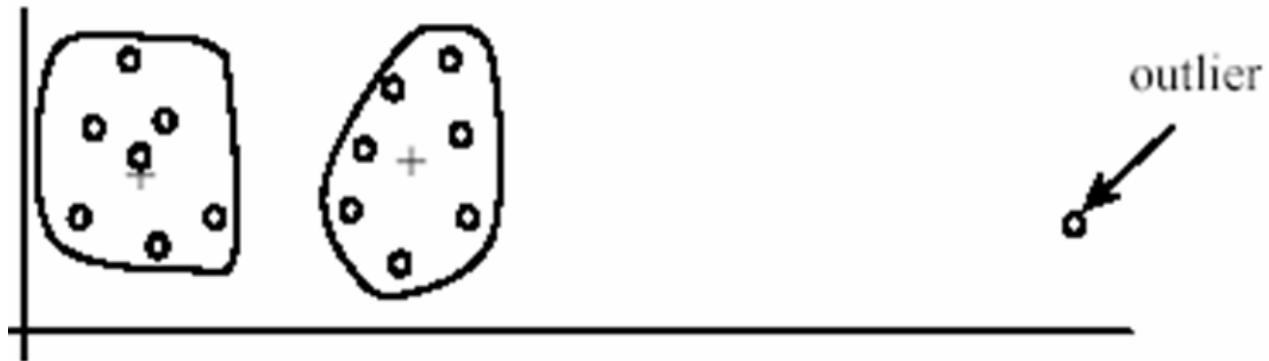
Partition Algorithm

1. Decide on a value for k .
2. Initialize the k cluster centers (randomly, if necessary).
3. Decide the class memberships of the N objects by assigning them to the nearest cluster center.
4. Re-estimate the k cluster centers, by assuming the memberships found above are correct.
5. If none of the N objects changed membership in the last iteration, exit. Otherwise goto 3.

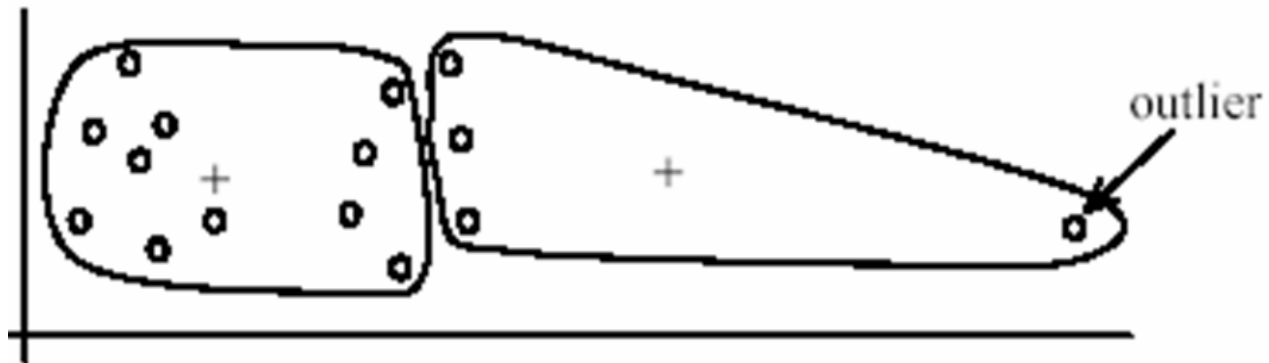
k-means converges to a local minimum



k-Means outliers problems



(B): Ideal clusters



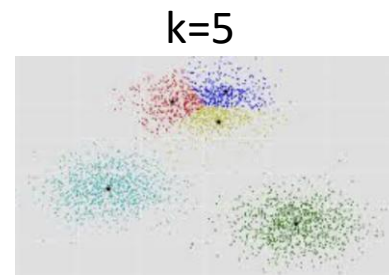
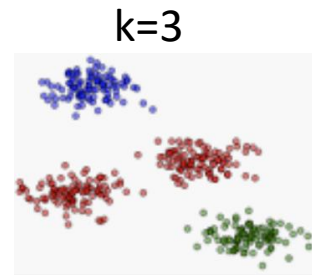
Strengths of k-Means

- *Relatively efficient: $O(tkn)$, where*
 - n is # objects,
 - k is # clusters,
 - t is # iterations.
 - Normally, $k, t \ll n$.
- Guarantees convergence
 - (even if terminates at a **local** minimum)
- Easily adapts to
 - New examples.
 - Generalizes to clusters of different shapes and sizes, such as elliptical clusters.

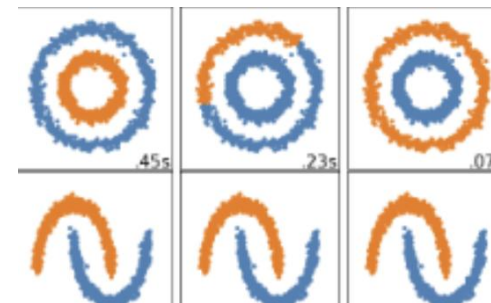
Weakness of k-Means

- Applicable only when **mean** is defined,
 - what about categorical data
("apple", 90mm), ("lemon", 45mm)
("orange", 55mm), ?

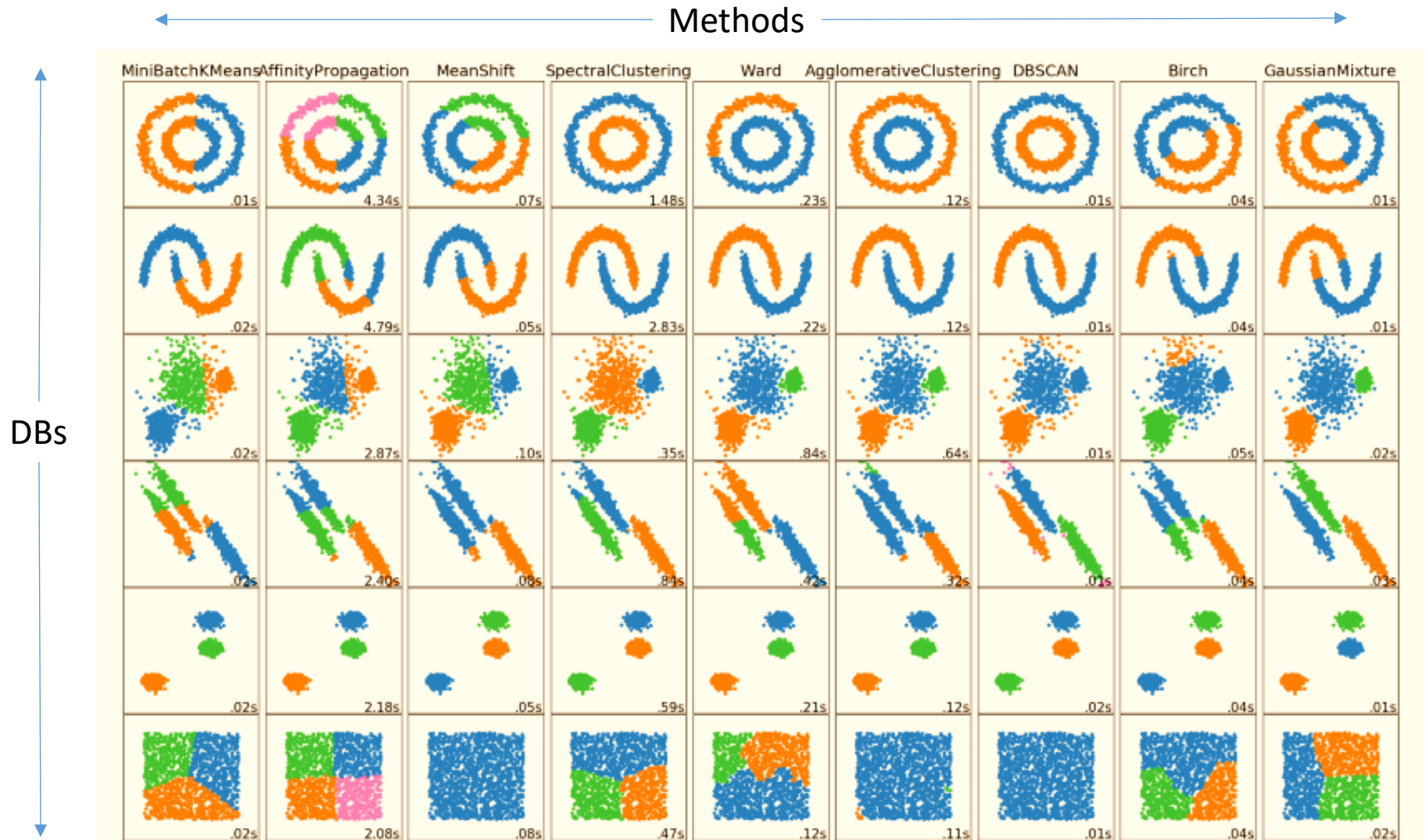
- **Need to specify k** ,
the number of clusters,
in advance



- Unable to handle noisy data and outliers
- Not suitable to discover clusters with **non-convex shapes**



Different behavior of the clustering methods



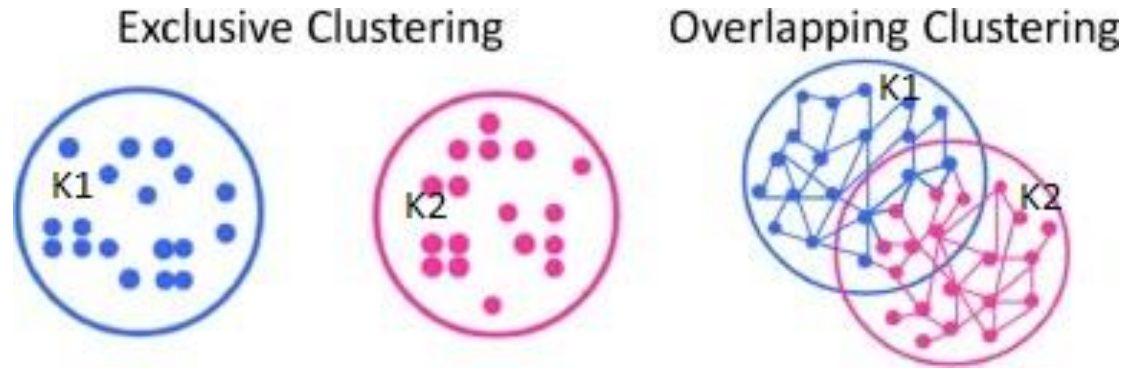
Exclusive and overlapping Clustering

Exclusive Clustering:

Each data point belongs to **exactly one** cluster with no ambiguity (e.g., K-Means); it creates distinct, non-overlapping partitions.

Overlapping Clustering:

Data points can hold membership in **multiple clusters** simultaneously (e.g., Fuzzy C-Means), reflecting real-world scenarios where categories intersect.



Key Distinction

While exclusive clustering forces a "hard" boundary, overlapping clustering allows for "soft" or shared membership based on different shared characteristics.

You have to choose w.r.t. your application

How to choose k the number of clusters

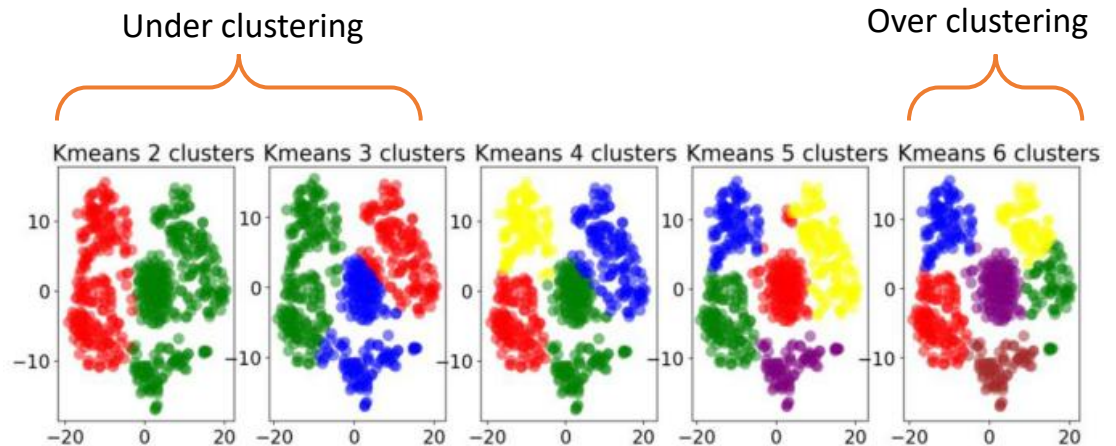
- A prior knowledge?
Use it!! 😊

- Too low k? Under clustering

Occurs when the number of clusters k is too low compared to the natural structure of the data.

- Too high k? Over clustering

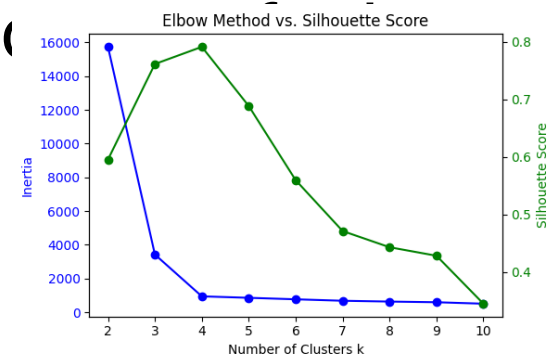
Occurs when the number of clusters k is too high, exceeding the actual number of natural groupings.



- Occurs when the number of clusters (k) is **too low** compared to the natural structure of the data.

Number of clusters k

- In unsupervised learning, the true number of clusters k is not known.
- We can fix the number of clusters to a small value to study the structure of the data and to compare different feature representations.
- There are advanced techniques to find the good k
 - the **elbow** method
 - The **silhouette** score



Pipeline

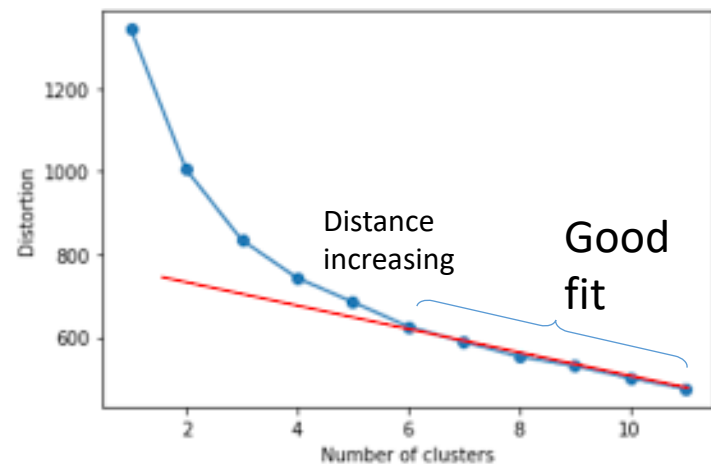
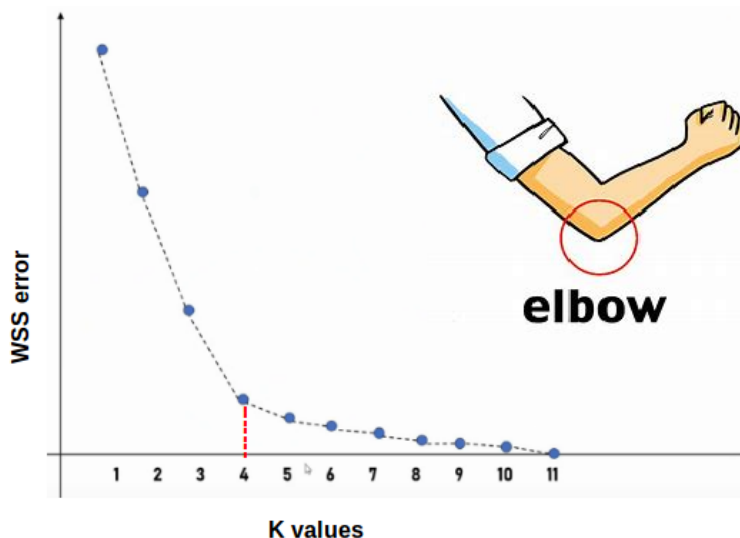
Raw data

- → Visual inspection
- → Encoding (categorical → numeric)
- → Scaling
- → Clustering (k-means)
 - Method, Metrics, Number K of clusters
- → PCA (visualization only)

How to decide k?

Elbow plot (basic method)

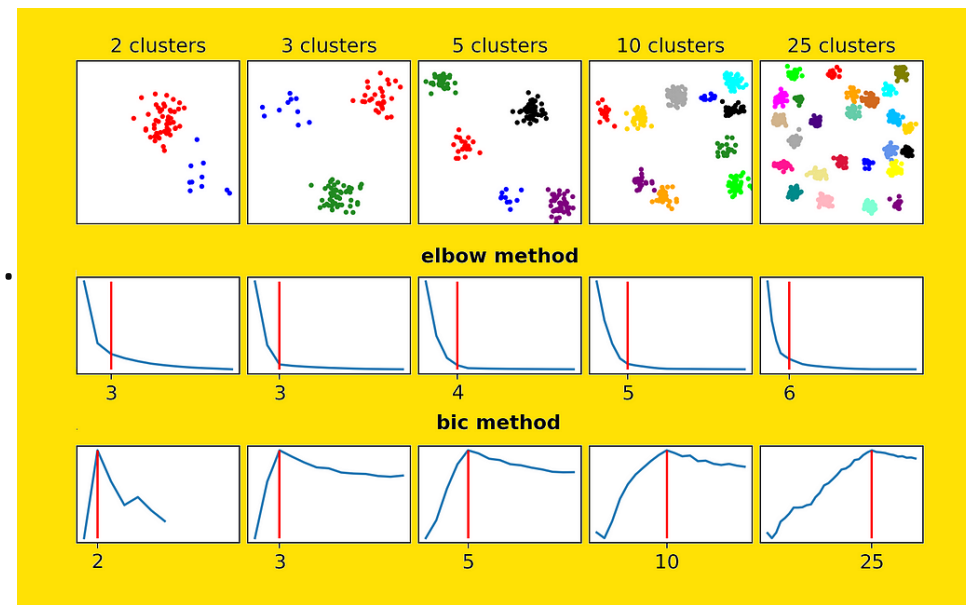
- The **elbow method** is a technique used to determine the optimal number of clusters in a dataset by plotting the **within-cluster sum of squares (WCSS)** against the number of clusters (k). The optimal number of clusters is identified at the "elbow point," where the rate of decrease in WCSS slows down, indicating diminishing returns on additional clusters. The elbow point can be identified from the points in the plot (e.g., by a linear regression of the last points).



How to decide k?

Advanced methods

- **Silhouette Score:**
Measures the coherence of clusters.
- **Gap Statistic:**
Compares with a null distribution.
- **Davies-Bouldin Index:**
Measures cluster separation and compactness.
- **BIC/AIC for GMM:**
Model selection criteria for Gaussian Mixture Models.
- **Calinski-Harabasz Index:**
Measures variance ratio for clusters.

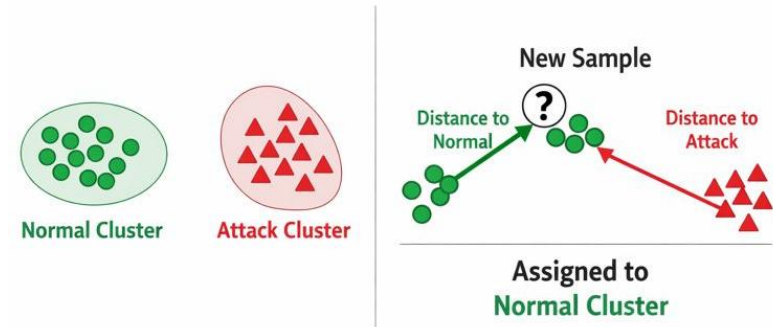


For ex., Bayesian Information Criterion (BIC) and Akaike Information Criterion (AIC) balance model fit and model complexity by penalizing the addition of more parameters

How to manage a new sample?

- Clustering algorithms do not directly classify new samples. They discover structure in the data, which must then be interpreted.
- A clustering model does **not assign semantic labels**
- New samples are **not “classified”**, but **positioned** in feature space
- The same preprocessing (features + scaling) must be applied
- Interpretation depends on **distance and proximity to clusters**
- **Same feature space, same metrics**

In clustering, we do not ask “*what class is this?*” but “*where does this sample belong?*”



The labels “**Normal Cluster**” and “**Attack Cluster**” are **assigned a posteriori** for interpretation and visualization purposes only. They are **not known to the clustering algorithm** and are **not used during the clustering process**

Assigning New Samples to Existing Clusters

Cluster-Based Classification

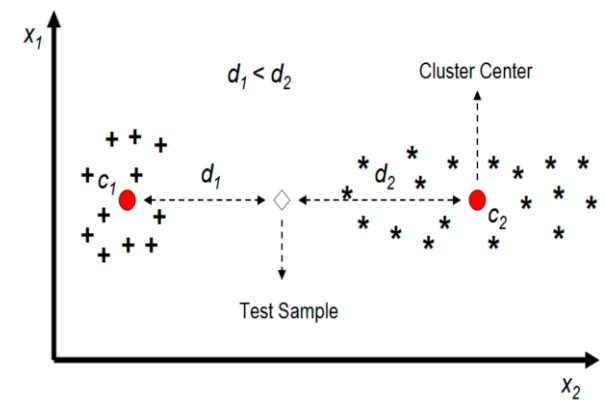
- Clusters summarize the structure of the dataset
- Each cluster is represented by a **centroid** (or prototype)

New Sample Assignment

- Given a new sample x_{new} :
 - Compute its distance to each cluster centroid
 - Assign it to the **closest cluster**
- where μ_k is the centroid of cluster k

$k (\leq n)$ sets $S = \{S_1, S_2, \dots, S_k\}$

$$\arg \min_{\mathbf{S}} \sum_{i=1}^k \sum_{\mathbf{x} \in S_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2$$



$$\hat{c} = \arg \min_k d(x_{new}, \mu_k)$$

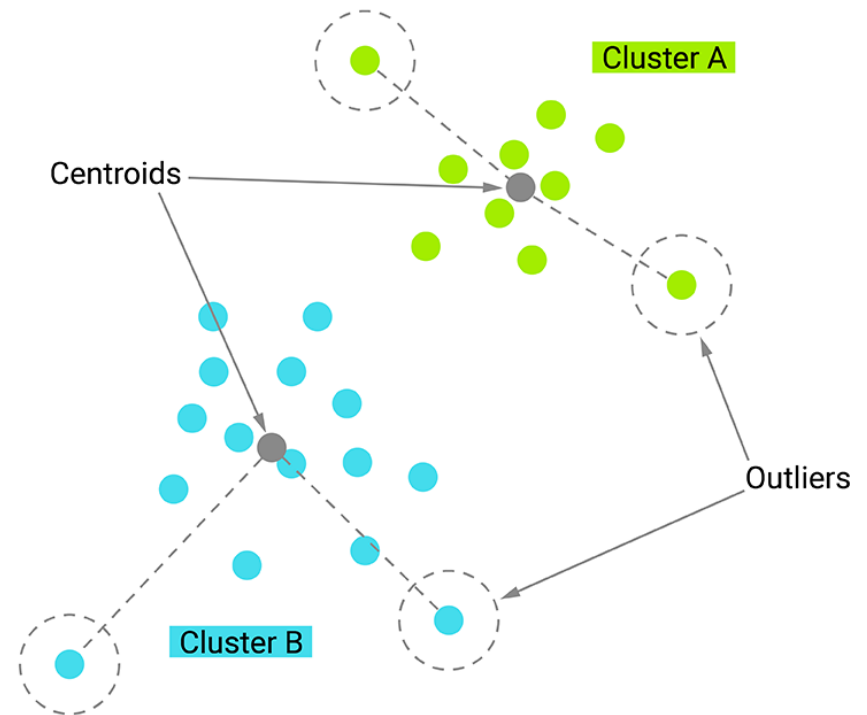
Clustering is not Anomaly Detection

Clustering:

- finds structure
- does NOT define anomalies

Anomalies are:

- low-density regions
- far from centroids
- small or isolated clusters



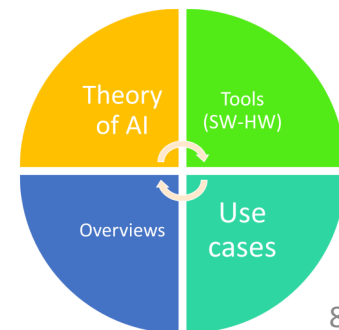


Toolboxes

EDA and Clustering in Python



A simple example
to review the techniques
we just studied



Colab k-means coding

```
[1] from google.colab import files
    uploaded = files.upload()
```

Scegli file Mall_Customers.csv

- **Mall_Customers.csv**(application/vnd.ms-excel) - 4286 bytes, last modified: 25/9/2019 - 100% done
Saving Mall_Customers.csv to Mall_Customers.csv

```
#import libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```



Mall_Customers.csv

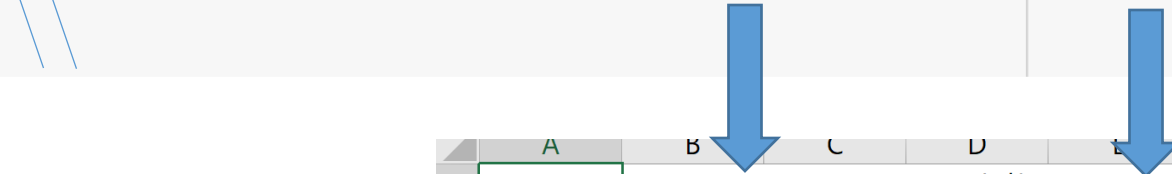
CustomerID,Genre,Age,Annual Income (k\$),Spending Score (1-100)				
1	CustomerID	Genre	Age	Annual Income (k\$),Spending Score (1-100)
2	0001	Male	19	15,39
3	0002	Male	21	15,81
4	0003	Female	20	16,6
5	0004	Female	23	16,77
6	0005	Female	31	17,40
7	0006	Female	22	17,76
8	0007	Female	35	18,6

pandas - Python Data Analysis Library

Import data and feature selection

```
#import data set
dataset = pd.read_csv('Mall_Customers.csv') # CustomerID,Genre,Age,Annual Income (k$),Spending Score (1-100)
print(dataset.shape)
#cut the dabase: only Age, Spending Score (1-100)
X = dataset.iloc[:,[2,4]].values
X[0:9]
```

```
(200, 5)
array([[19, 39],
       [21, 81],
       [20, 6],
       [23, 77],
       [31, 40],
       [22, 76],
       [35, 6],
       [23, 94],
       [64, 3]])
```



	A	B	C	D	E	F
1	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)	
2	0001	Male	19	15	39	
3	0002	Male	21	15	81	
4	0003	Female	20	16	6	
5	0004	Female	23	16	77	
6	0005	Female	31	17	40	
7	0006	Female	22	17	76	
8	0007	Female	35	18	6	

Let's image that in your project you have to focus
on **Age** and the **Spending Score**

Data exploration 1 .head



#DATA EXPLORATION

```
dataset.head(10)
```

	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40
5	6	Female	22	17	76
6	7	Female	35	18	6
7	8	Female	23	18	94
8	9	Male	64	19	3
9	10	Female	30	19	72

Data exploration 2 .shape



```
dataset.shape
```

(200, 5)



check



	A	B	C	D	E	F
1	CustomerID	Genre	Age	Annual Income (k\$)	Spending Score (1-100)	
2	0001	Male	19	15	39	
3	0002	Male	21	15	81	
4	0003	Female	20	16	6	
5	0004	Female	23	16	77	
6	0005	Female	31	17	40	
7	0006	Female	22	17	76	
8	0007	Female	35	18	6	



```
dataset.describe().T
```

	count	mean	std	min	25%	50%	75%	max
CustomerID	200.0	100.50	57.879185	1.0	50.75	100.5	150.25	200.0
Age	200.0	38.85	13.969007	18.0	28.75	36.0	49.00	70.0
Annual Income (k\$)	200.0	60.56	26.264721	15.0	41.50	61.5	78.00	137.0
Spending Score (1-100)	200.0	50.20	25.823522	1.0	34.75	50.0	73.00	99.0

Data exploration 3 .info



```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 200 entries, 0 to 199
```

```
Data columns (total 5 columns):
```

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	CustomerID	200 non-null	int64
1	Genre	200 non-null	object
2	Age	200 non-null	int64
3	Annual Income (k\$)	200 non-null	int64
4	Spending Score (1-100)	200 non-null	int64

```
dtypes: int64(4), object(1)
```

```
memory usage: 7.9+ KB
```

Data exploration 4 .isnull

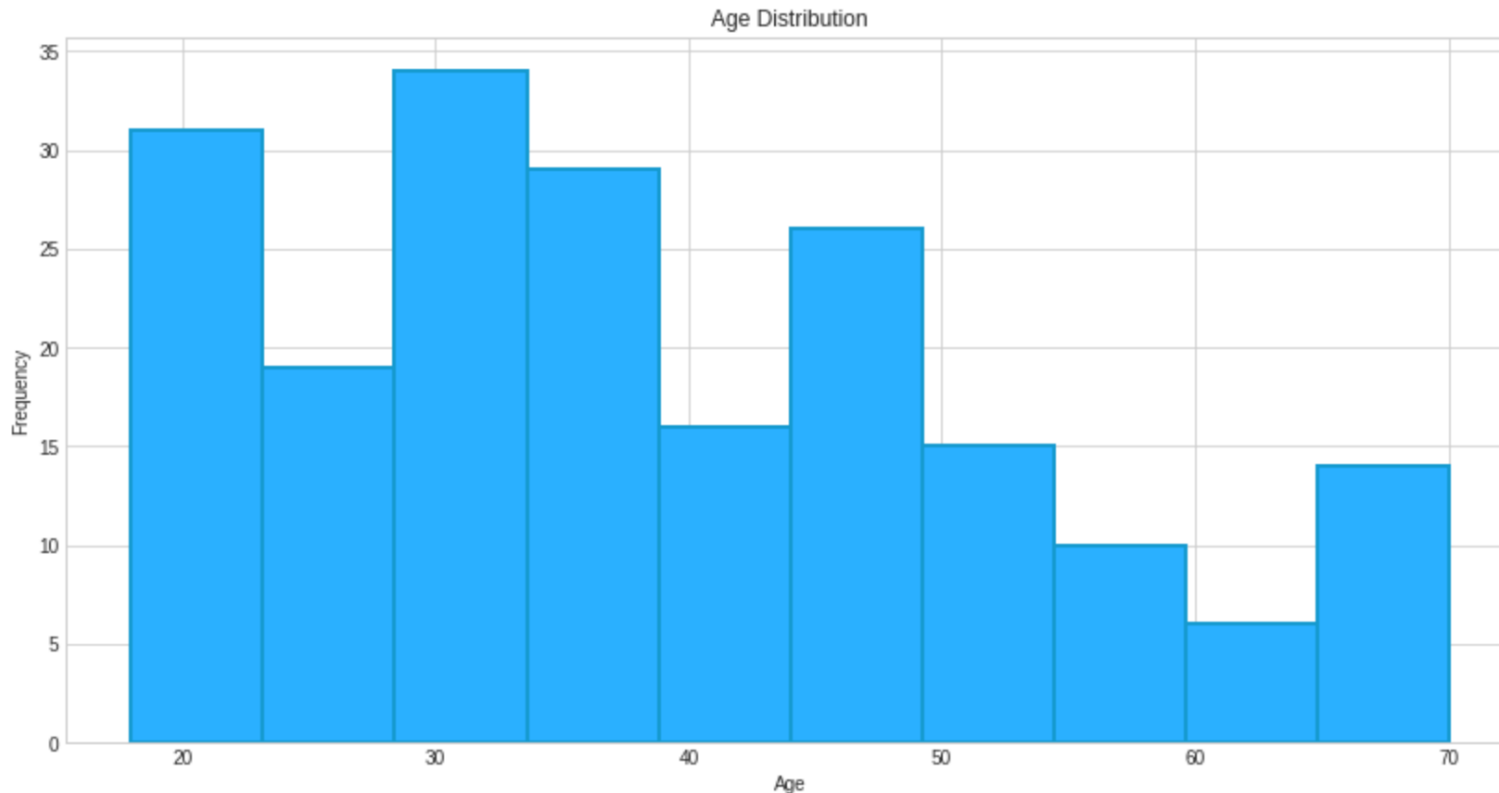


```
dataset.isnull().sum()
```

```
↳ CustomerID      0
   Genre           0
   Age             0
   Annual Income (k$)  0
   Spending Score (1-100)  0
   dtype: int64
```

Data exploration 5 plt.hist

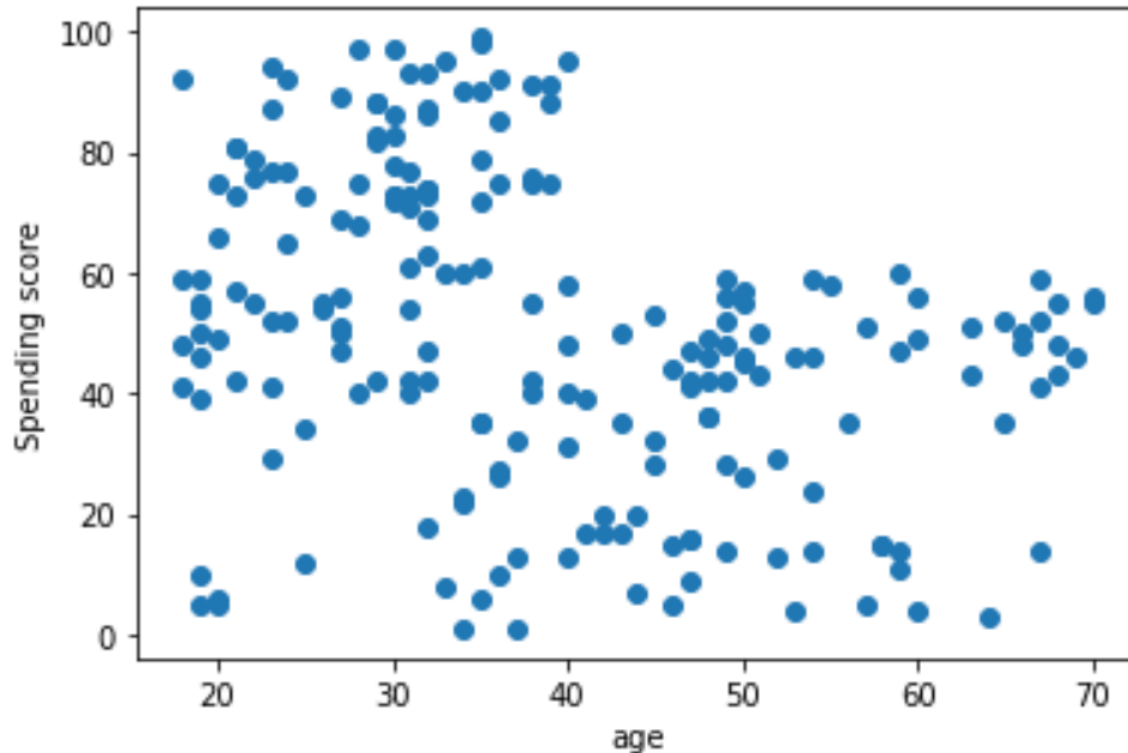
```
# age distribution
plt.style.use('seaborn-whitegrid')
plt.figure(figsize=(14,7))
plt.hist(dataset['Age'], bins=10, facecolor = '#2ab0ff', edgecolor='#169acf', linewidth=2)
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Frequency')
plt.show()
```



Plotting of the data

```
plt.xlabel('age')  
plt.ylabel('Spending score')  
plt.scatter(X[:, 0], X[:, 1])
```

<matplotlib.collections.PathCollection at 0x7f5bb4909710>

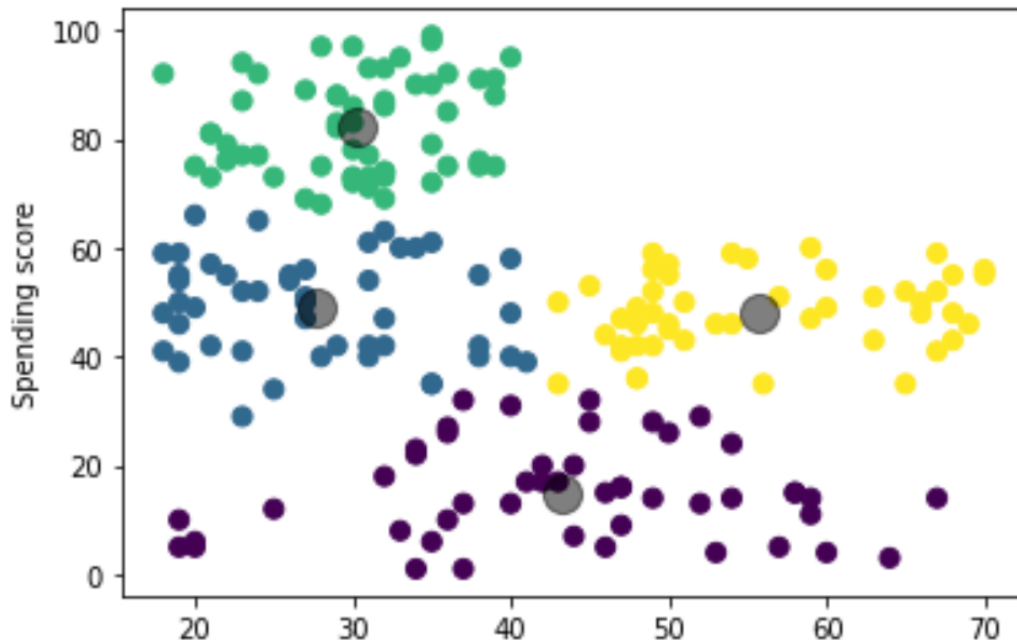


```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=4)
kmeans.fit(X)
y_kmeans = kmeans.predict(X)
centers = kmeans.cluster_centers_
```

k-means

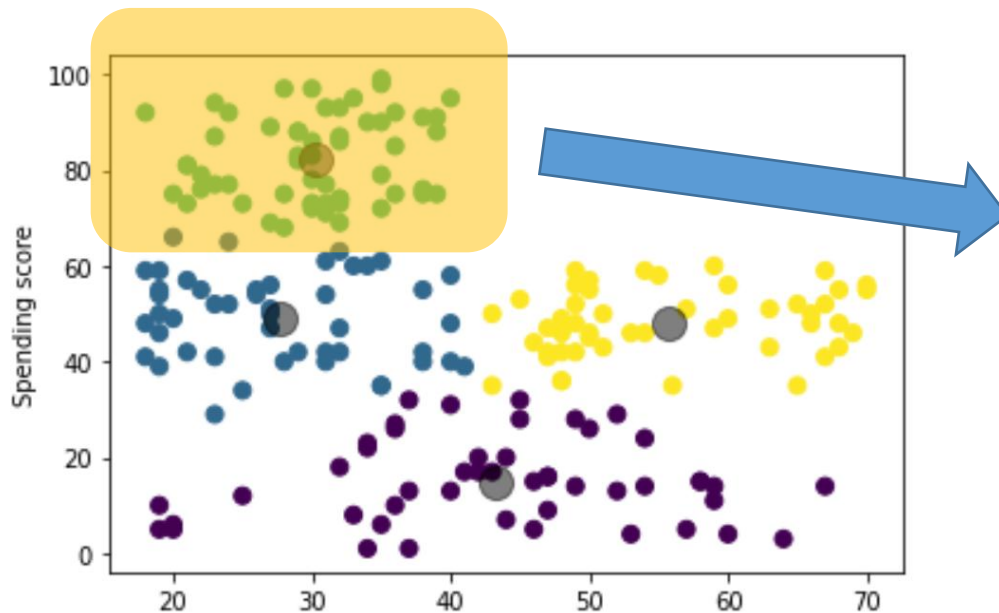
```
plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis')
plt.scatter(centers[:, 0], centers[:, 1], c='black', s=200, alpha=0.5);
plt.xlabel('age')
plt.ylabel('Spending score')
```

```
Text(0, 0.5, 'Spending score')
```



In >3D the clustering
can help even more...

Clustering can be used for insights!



We identified a cluster characterized by

- young age
- high spending score

Send often coupons and dedicated offers!!

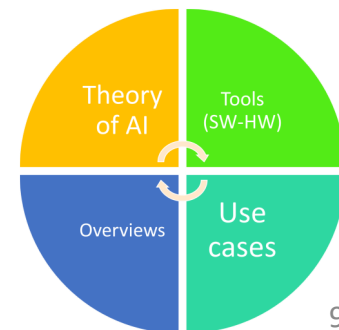


Toolboxes

EDA and Clustering in Python



An example coming from an industrial application:
signals from real sensors

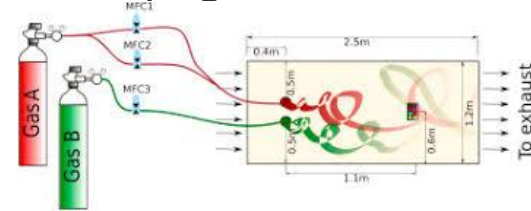


Clustering signals

(not in the exam, but a good review)

gas_clustering.py

This data set contains the acquired time series from 16 chemical sensors exposed to gas mixtures at varying concentration levels.



ethylene_methane2_red.txt

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

# Constants
N_LINES_UNDERFITTING = 50

# Define the path to the local file
file_path = 'ethylene_methane2_red.txt'

# Read the dataset from the local file
try:
    # Read the file with whitespace delimiter and handle
    with open(file_path, 'r') as file:
        lines = file.readlines()

    # Skip the header and split each line into columns
    data = []
    for line in lines[1:]:
        data.append([float(value) for value in line.split()])

    # Convert the list to a DataFrame
    columns = ['Time', 'Methane_conc', 'Ethylene_conc']
    data = pd.DataFrame(data, columns=columns)

    # Downsampling the data: keep 1 out of every N_LINES_UNDERFITTING
    data = data.iloc[::N_LINES_UNDERFITTING, :]

except Exception as e:
    print("Error parsing the file:", e)
else:
    # Exploratory Data Analysis (EDA)
```

Time (seconds)	Methane conc (ppm)	Ethylene conc (ppm)	sensor readings (16 channels)															
0.00	0.00	0.00	-41.98	2067.64	-37.13	2.28	8.63	-26.62	-8.46	-0.33	3437.73	2728.14	4054.03	4007.89	4478.27	5056.98	3639.09	3128.49
1.00	0.00	0.00	-41.17	2081.41	2.77	56.30	28.40	4.23	21.86	34.95	3435.38	2723.39	4040.22	4017.80	4494.33	5046.29	3635.46	3131.01
2.00	0.00	0.00	59.43	2072.86	198.53	247.17	186.35	228.07	233.69	288.70	3451.27	2729.19	4023.24	3988.75	4497.06	5089.52	3647.27	3128.21
3.00	0.00	0.00	364.55	2070.96	689.49	778.38	701.60	1003.06	746.90	877.61	3436.85	2736.05	4013.64	4000.23	4512.81	5048.86	3651.21	3133.82

Columns:

Time (seconds)

Methane conc (ppm)

Ethylene conc (ppm)

Sensor readings (16 channels)

16 Channels → 16 Dimensions feature space

Clustering signals: EDA

(not in the exam)

Exploratory Data Analysis (EDA):

Head of the dataset:

	Time	Methane_conc	Ethylene_conc	...	Sensor_14	Sensor_15	Sensor_16
0	0.00	0.00	0.00	...	5056.98	3639.09	3128.49
50	50.00	0.00	0.00	...	1482.40	1756.83	1436.23
100	100.01	133.33	0.00	...	764.41	1185.96	904.64
150	150.27	133.33	0.00	...	2649.78	1855.28	1650.49
200	200.27	0.00	16.67	...	2712.34	1892.00	1660.66

[5 rows x 19 columns]

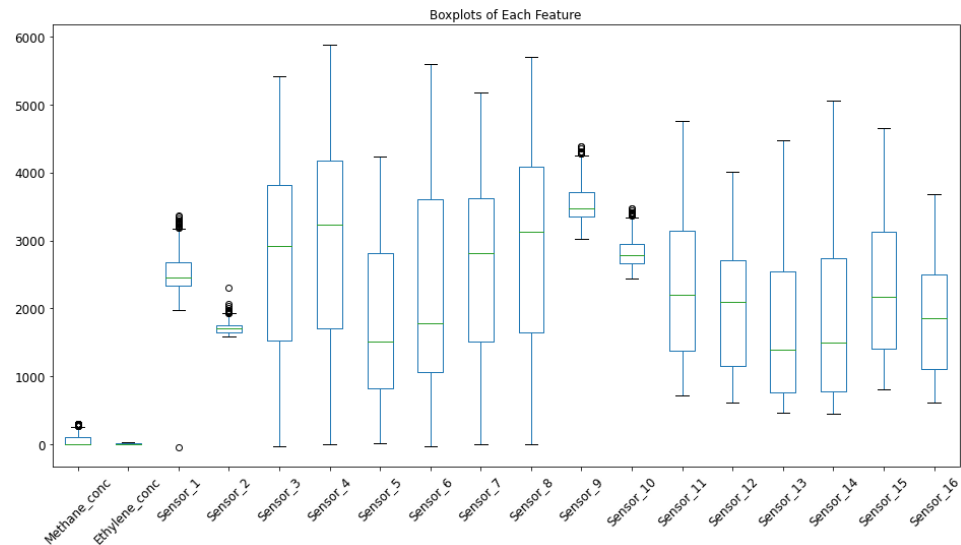
Dataset type:

The dataset is unsupervised as it does not include explicit target variables.

Missing values and percentages:

Time 0.0
Methane_conc 0.0
Ethylene_conc 0.0
Sensor_1 0.0
Sensor_2 0.0
Sensor_3 0.0
Sensor_4 0.0
.....

dtype: float64

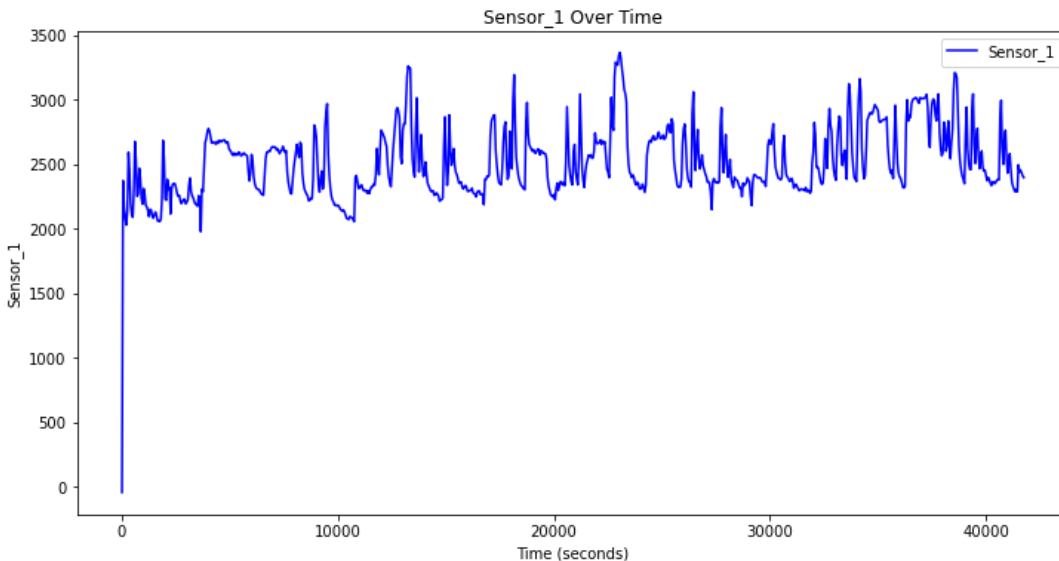


Clustering signals: EDA

(not in the exam)

```
# Dropping columns that are not sensor features (Time, Methane_conc, Ethylene_conc)
features_to_drop = ['Time', 'Methane_conc', 'Ethylene_conc']
data = data.drop(columns=features_to_drop)
```

Plotting the first sensor



Scatter plot S3 vs S6

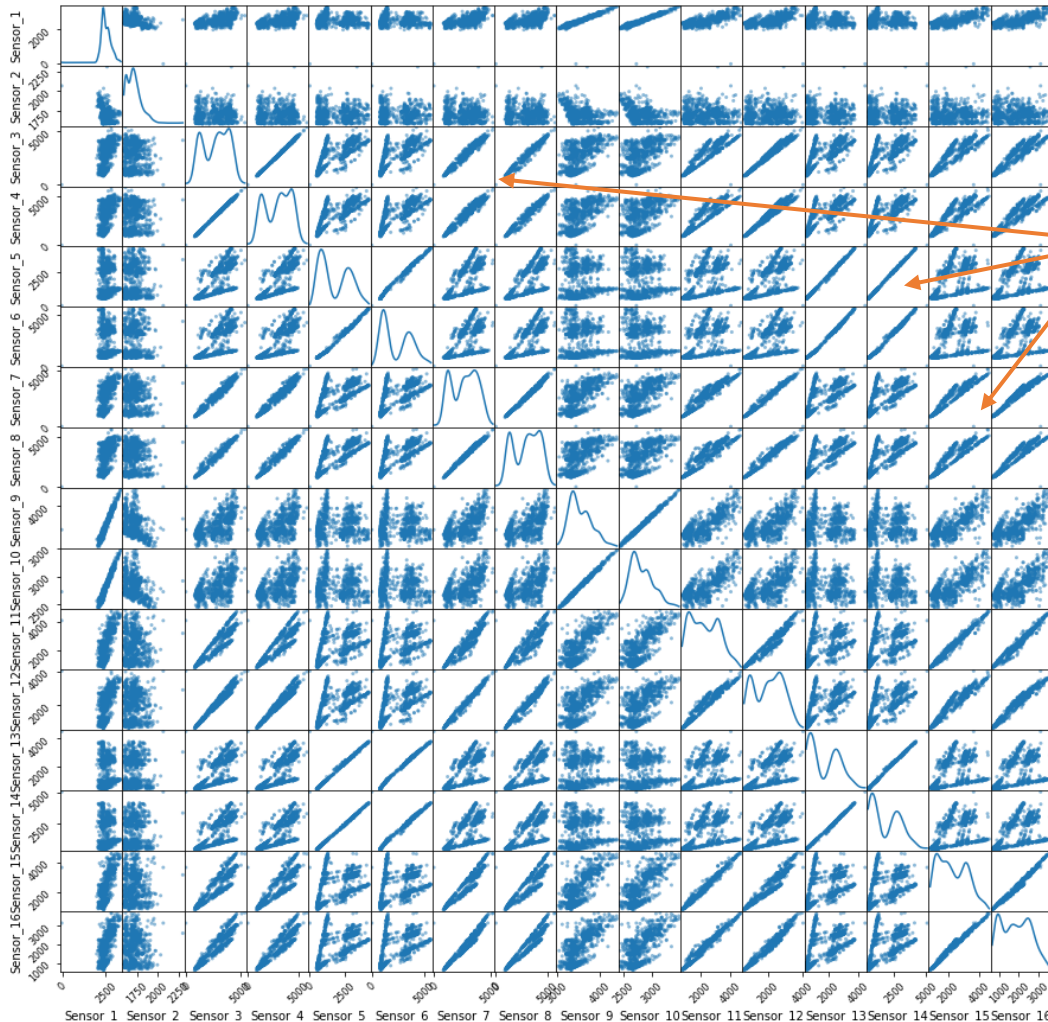


Clustering signals: EDA

(not in the exam)

Scatter Matrix of Sensor Features

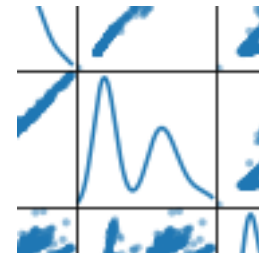
We have to explore a 16 dimensions space...



Always a good idea to check the distributions of the features

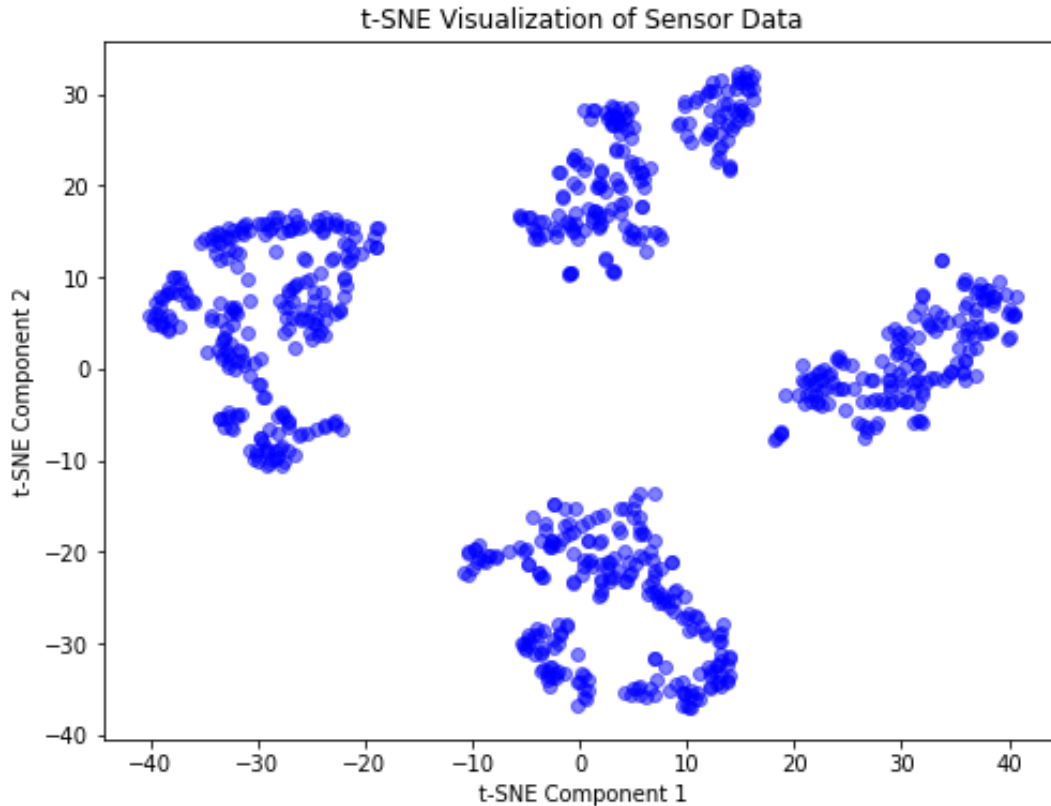
We can see sensors very correlated

Bimodal or N model distributions are a good hint for “states” in the systems



Clustering signals: EDA

(not in the exam)



The existence of “states” in the systems is confirmed in the t-SNE visualization (what powerful unsupervised embedding transformation!)

Clustering signals: EDA

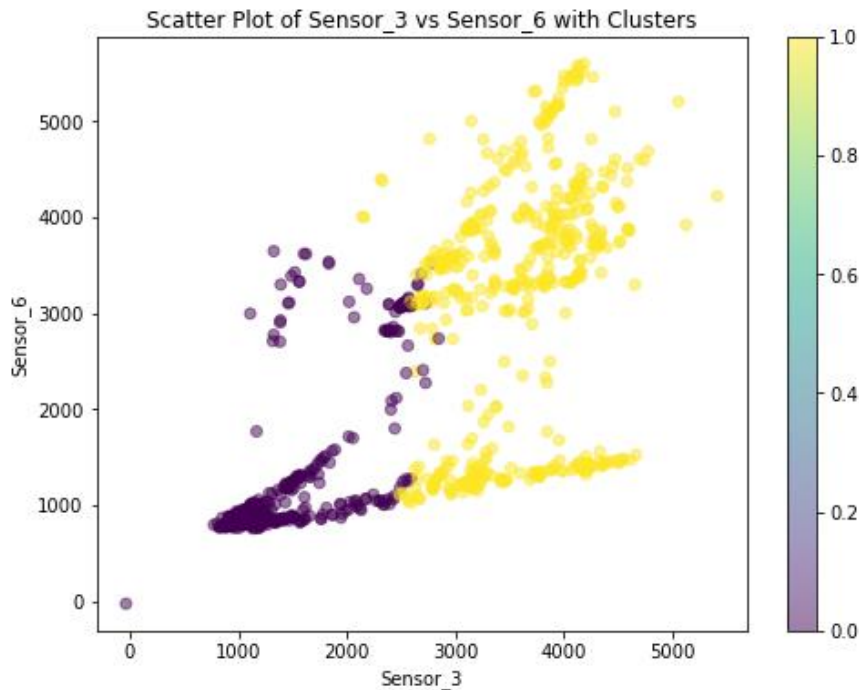
(not in the exam)

```
scaler = StandardScaler()  
data_scaled =   
scaler.fit_transform(data)
```

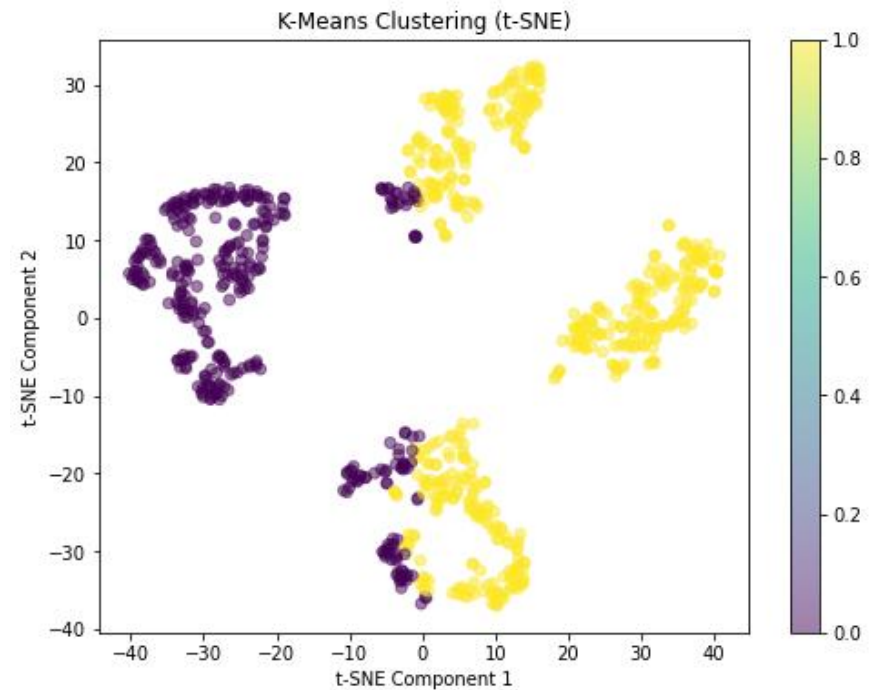
Applying K-Means Clustering **(in the standardized feature space)**

```
kmeans = KMeans(n_clusters=2, random_state=42)
```

```
kmeans_labels = kmeans.fit_predict(data_scaled)
```



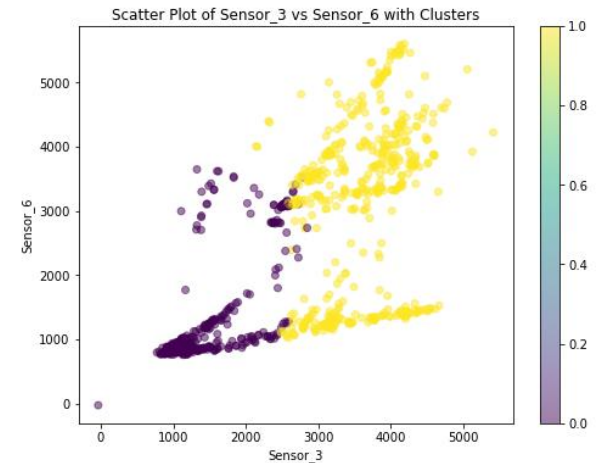
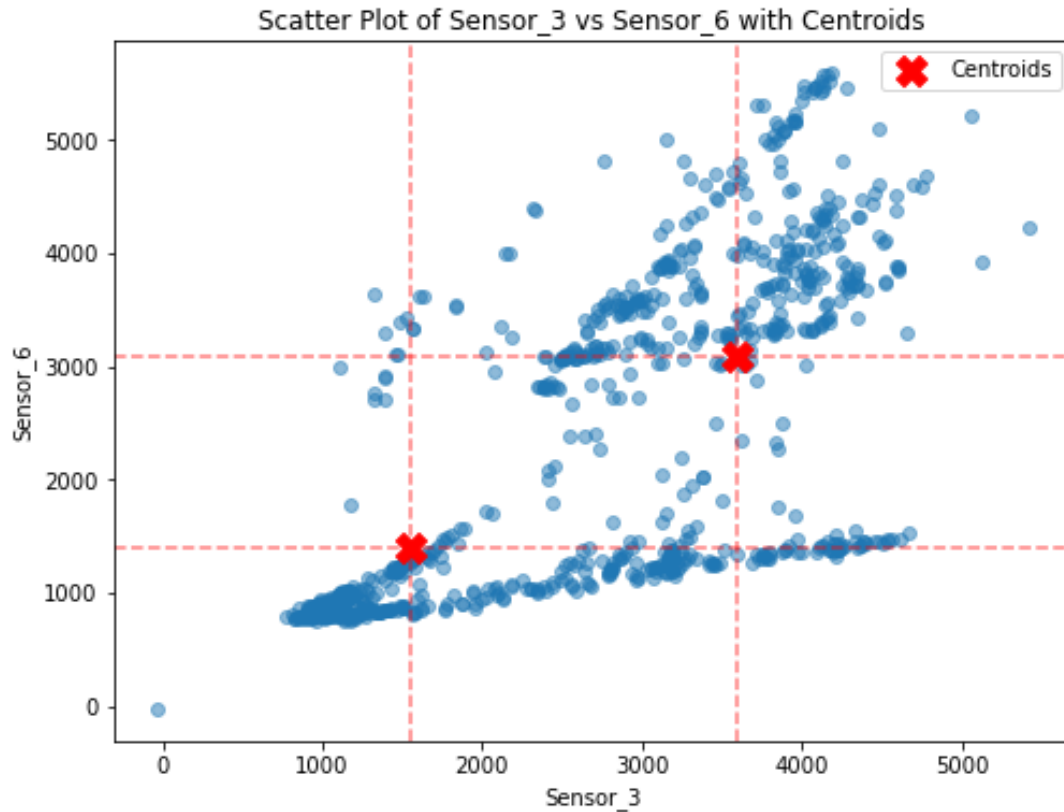
Scatter plot S3 vs S6
+ color labelling according to the clusters



T-SNE comp. 1 VS 2
+ color labelling according to the clusters

Clustering signals: EDA

(not in the exam)

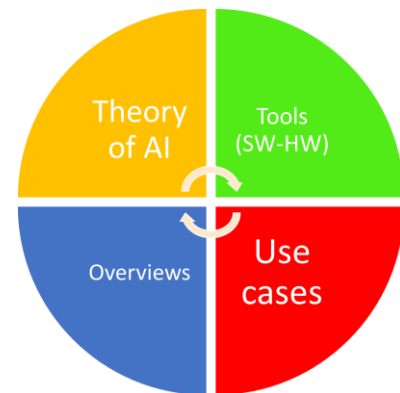


Of course you can do the clustering in the t-SNE components



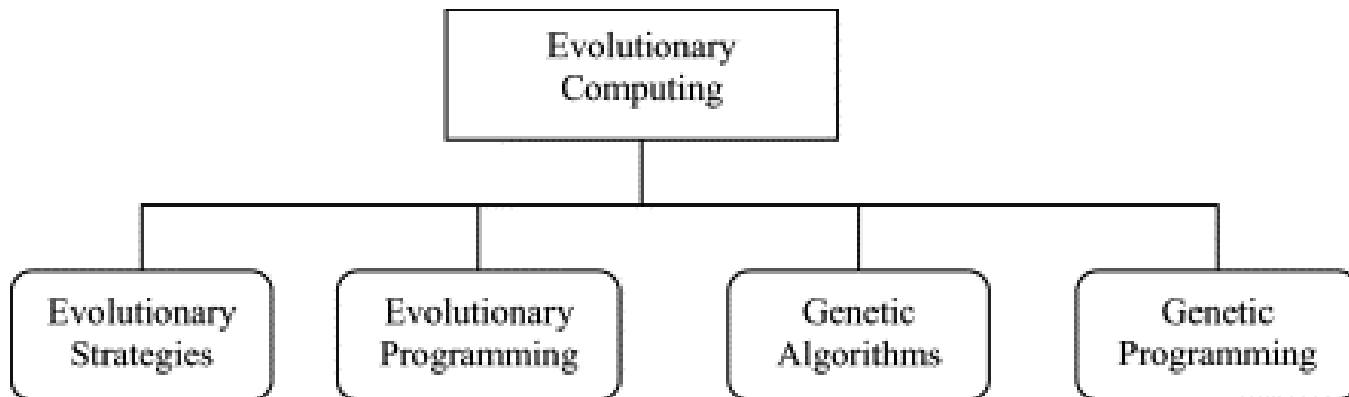
THEORY

Evolutionary computing



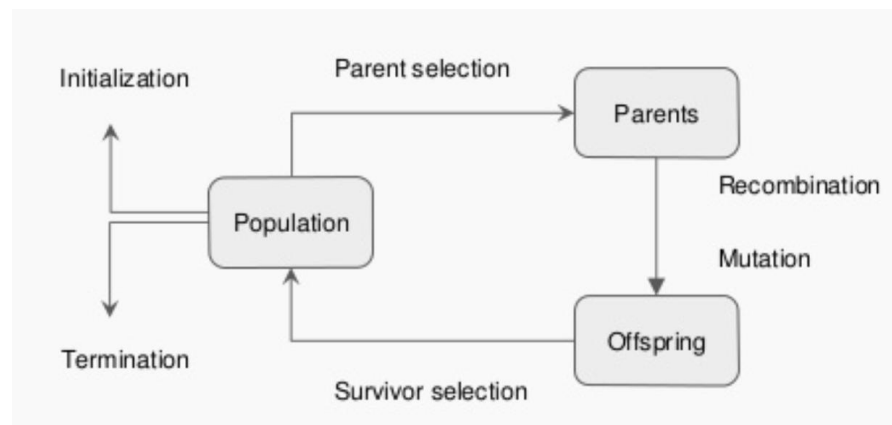
Genetic Algorithms (1)

- Based on Darwin's evolutionary principle of “survival of the fittest”
- GAs require the ability to recognize a good solution, but not how to get to that solution



Genetic Algorithms (2)

- The procedure:
 - An initial set of random solutions is ranked in terms of ability to solve the problem at hand
 - The best solutions are then crossbred and mutated to form a new set
 - The ranking and formation of new solutions is continued until a good enough solution is found or ...



Fuzzy vs Boolean Logic

Boolean Logic

Is Nick Honest?

True/Yes/1

False/No/0

Fuzzy Logic

Is Nick Honest?

Extremely Honest

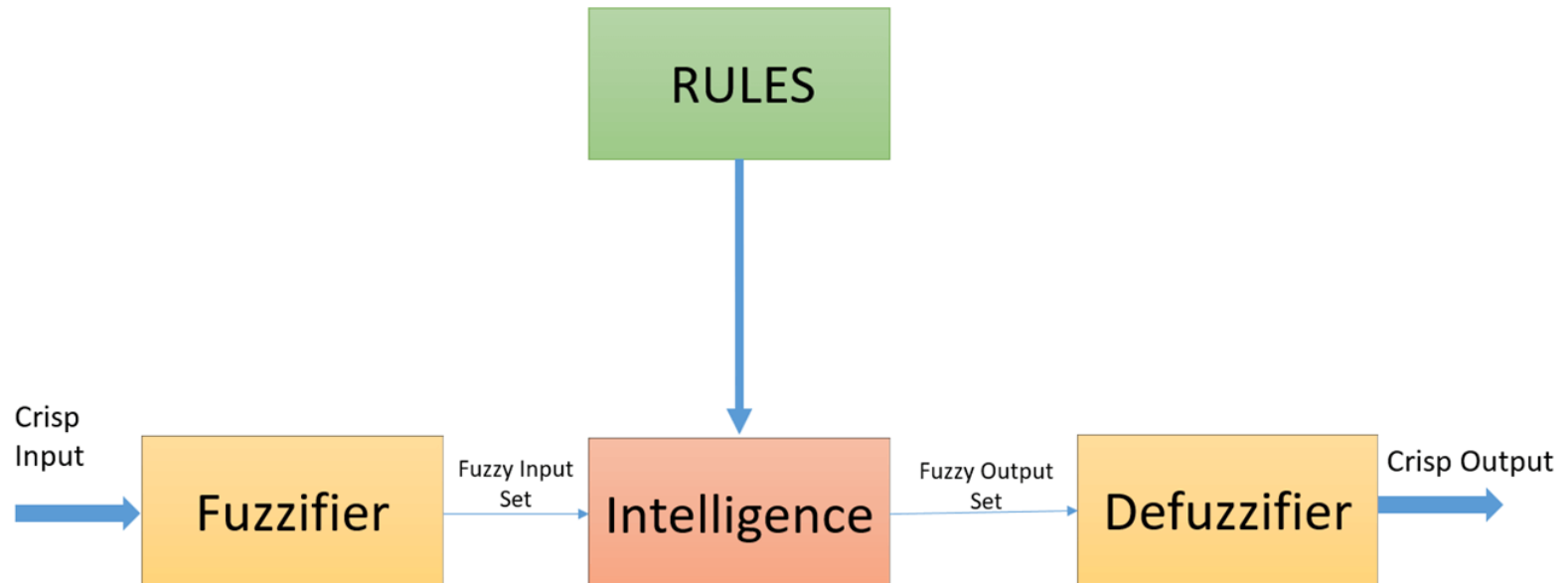
Very Honest(0.85)

Sometimes Honest(0.35)

Extremely Dishonest(0.00)

Fuzzy Logic (cont.)

- Parts of the knowledgebase of a fuzzy system:
 - Fuzzy rules
 - Fuzzy sets



Swarm Intelligence Systems

- Inspired by the behavior of social insects such as ants, as well as other animal societies like flocks of birds or schools of fish.
- Characterized by
 - decentralized and self-organized behaviors
 - complex problem-solving
 - optimization capabilities

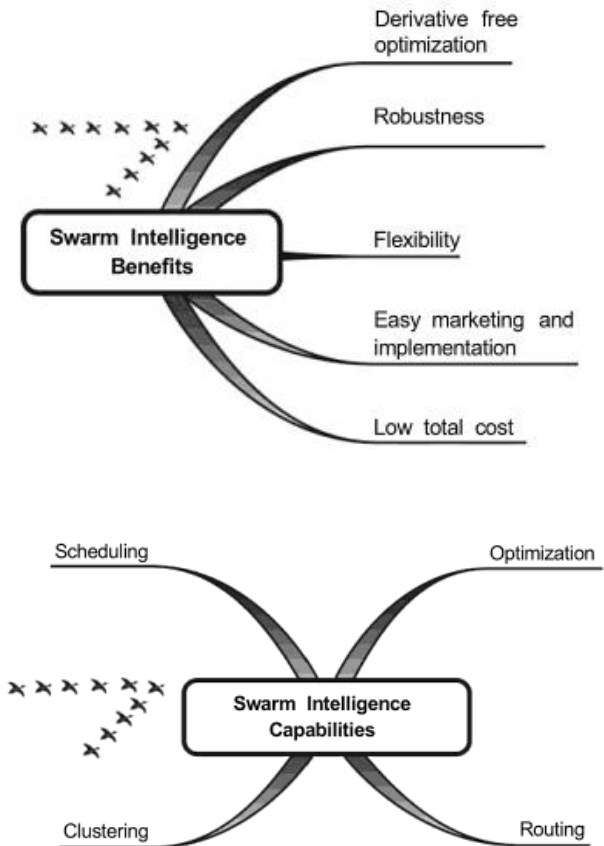
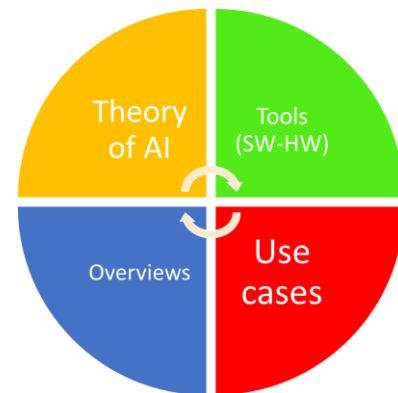
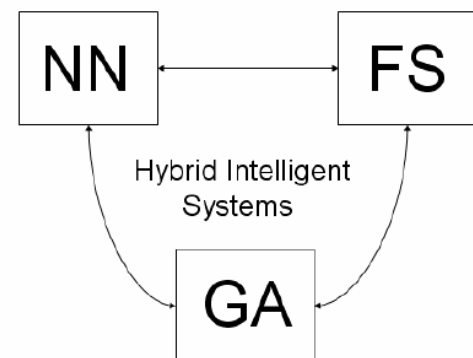


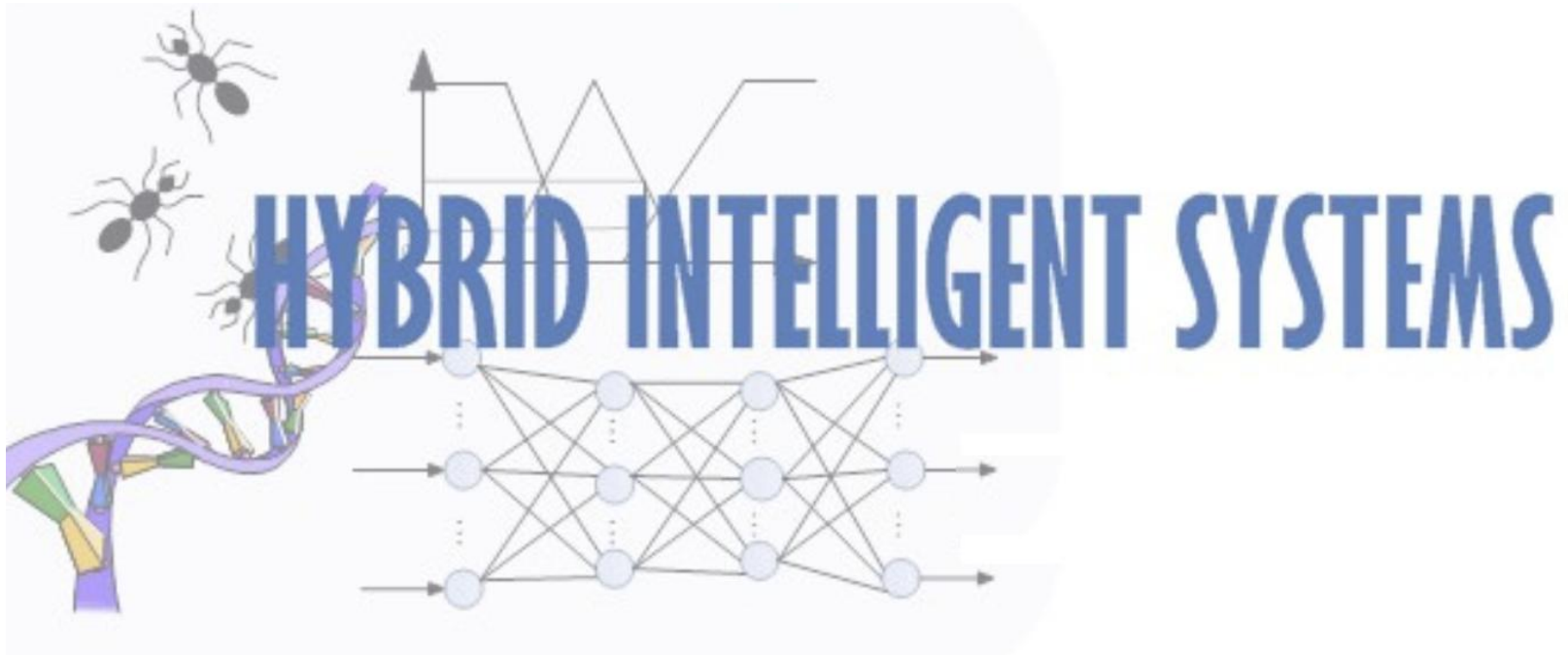
Fig. 6.11 Key capabilities of swarm intelligence



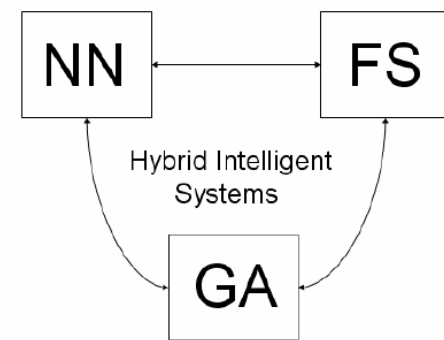
THEORY

Hybrid Intelligent Systems





“A software system which employs, in parallel, a combination of methods and techniques from artificial intelligence”

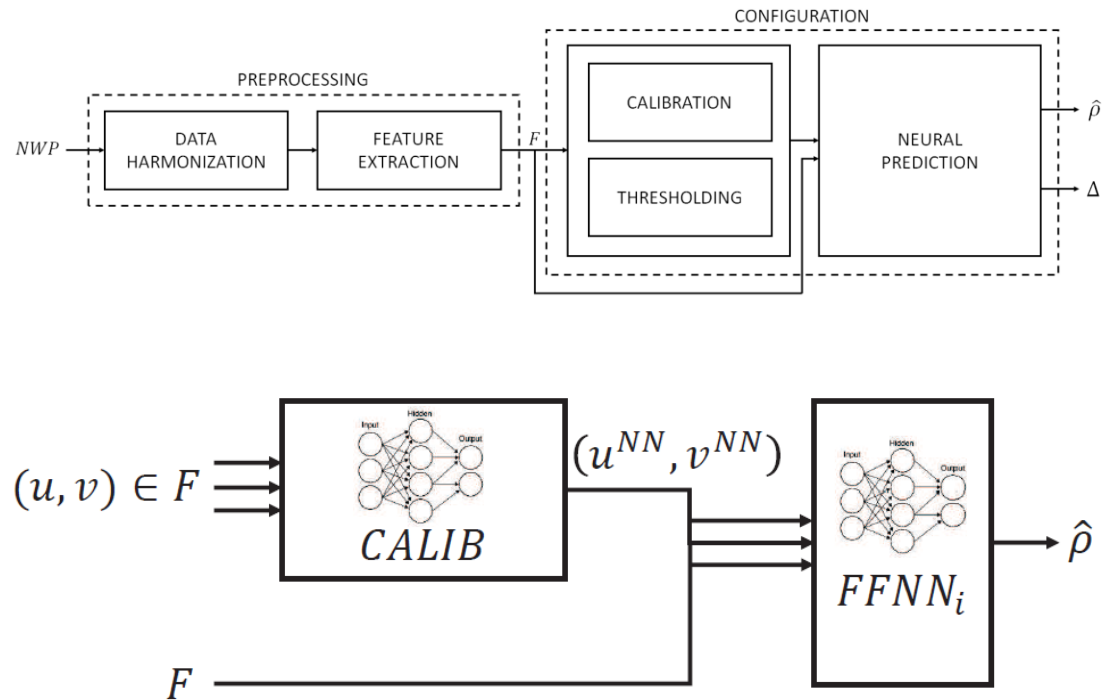
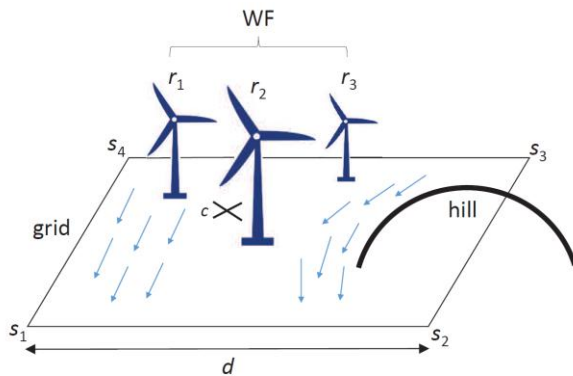
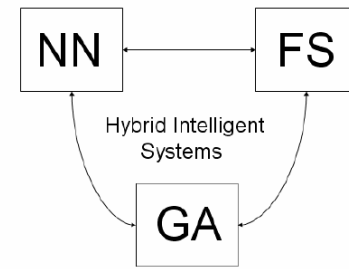


Why hybrid?

- Neural computing, machine learning, fuzzy logic, evolutionary algorithms, swarm intelligence, agent-based methods, among others (“soft computing”), have been established and shown their strength and drawbacks.
- Find the best combination of techniques to **mitigate the drawbacks** in **your** application
- Some possible fusions
 - Soft Computing/AI and hard computing
 - Different soft computing/AI
 - Embedding (e.g. neurofuzzy)

Examples of simple Hybrid Intelligent System

- They are just “in parallel” (non embedded)

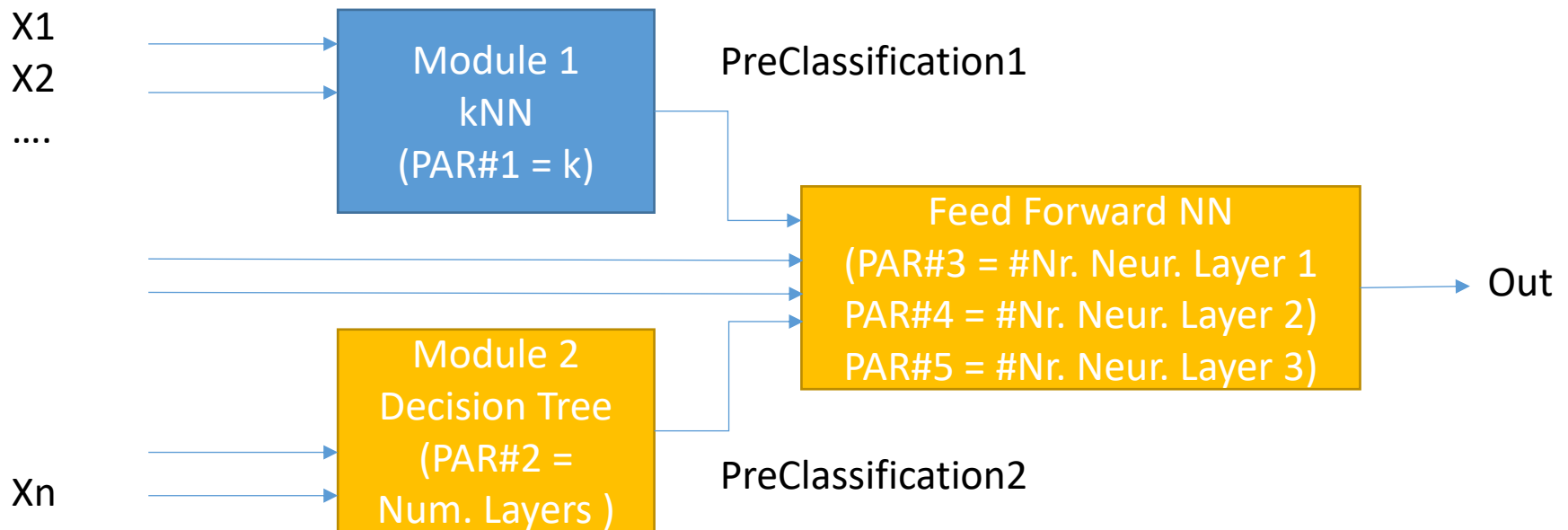


Examples of combinations

- Neuro-Fuzzy Systems
- Genetic-Fuzzy Systems
- Genetic-Neural Systems
- Neuro-Fuzzy-Genetic Systems
- Hybrid/Neural Expert Systems
- Swarm Intelligence Systems
- ... and many others

Example of Genetic Parameters Optimization

Even only 4-5 parameters in the project (such as in a hybrid system) can make the design impossible to be solved by brute force
→ Gen. Algor. can be a good solution



Example of Genetic Parameters Optimization

Population creation and selection of the fittest (or the N fittest)

Individual	Par.#1	Par.#2	Par.#3	Par.#4	Par.#5	Fitness (Accuracy)
#1	1	4	10	10	8	3.3%
#2	1	4	8	8	6	3.1%
#3	3	4	8	6	6	4.1%
...						...

Fittest!

Genetic Parameters Optimization: Crossover and mutations

Crossover point



Individual	Par.#1	Par.#2	Par.#3	Par.#4	Par.#5	Fitness (Accuracy)
#1 (Mother)	1	4	10	10	8	3.3%
#2 (Father)	1	4	8	8	6	3.1%
#12 (Child)	3	4	10	10	8	2.8%
#12 (Mutated Child)	3	4	10	3	8	2.8%



Mutation of random position
and intensity = (Par.#4, -5)

Main points



- Bayes Optimal Classification
- Pros and cons of Eager and Lazy Learning Methods
- Nearest Neighbor Classifiers (kNN)
 - Relevance of the kNN in pattern recognition
 - Approximator of the optimal Bays classifier
 - Definition and coding
 - Problems
 - Speed
- The KNN as “debug tool”
- The concept that classification is a set of boundaries in the feature space (e.g. Voronoi)

Main points 2



- Clustering (k means)
- Main types of computational intelligence systems
 - Fuzzy systems
 - Evolutionary systems
 - Hybrid systems
- Example of Genetic Parameters Optimization

